KNOWLEDGE-AUGMENTED LARGE LANGUAGE MODELS FOR ARABIC POST-OCR ERROR CORRECTION

By

Engineer's Name

A Thesis Submitted to the
Faculty of Engineering at Cairo University
in Partial Fulfillment of the
Requirements for the Degree of
MASTER OF SCIENCE
in
Electronics and Communications Engineering

FACULTY OF ENGINEERING ,CAIRO UNIVERSITY
GIZA,EGYPT
YYYY

KNOWLEDGE-AUGMENTED LARGE LANGUAGE MODELS FOR ARABIC POST-OCR ERROR CORRECTION

By

Engineer's Name

A Thesis Submitted to the
Faculty of Engineering at Cairo University
in Partial Fulfillment of the
Requirements for the Degree of
MASTER OF SCIENCE
in
Electronics and Communications Engineering

Under the Supervision of

Dr./Prof. Dr. Main Supervisor
Affiliation
Electronics and Communications Engineering
Faculty of Engineering , Cairo University

Dr./Prof. Dr. Co-supervisor
Affiliation
Department
Faculty, University

Dr./Prof. Dr. Co-supervisor
Affiliation
Department
Faculty, University

FACULTY OF ENGINEERING ,CAIRO UNIVERSITY
GIZA,EGYPT
YYYY

KNOWLEDGE-AUGMENTED LARGE LANGUAGE MODELS FOR ARABIC POST-OCR ERROR CORRECTION

By

Engineer's Name

A Thesis Submitted to the
Faculty of Engineering at Cairo University
in Partial Fulfillment of the
Requirements for the Degree of
MASTER OF SCIENCE
in
Electronics and Communications Engineering

Approved by the Examining Committee:

Dr./Prof. Dr. Main Supervisor,	Thesis Main Advisor
Dr./Prof. Dr. Int. Examiner,	Internal Examiner
Dr./Prof. Dr. Ext. Examiner, Affiliation Faculty, University	External Examiner

FACULTY OF ENGINEERING ,CAIRO UNIVERSITY
GIZA,EGYPT
YYYY

Engineer's Name: Engineer's Name
Date of Birth: dd/mm/yyyy
Nationality: Engineer's Nationality
E-mail: Engineer's E-mail
Phone: Engineer's Phone Number
Address: Street, City, Province, Zip Code
Registration Date: dd/mm/yyyy
Awarding Date: –/–/yyyy
Degree: Master of Science
Department: Electronics and Communications Engineering

Supervisors:

Dr./Prof. Dr. Main Supervisor
Dr./Prof. Dr. Co-supervisor
Dr./Prof. Dr. Co-supervisor

Examiners:

Dr./Prof. Dr. Main Supervisor (Thesis Main Advisor)
Dr./Prof. Dr. Int. Examiner (Internal Examiner)
Dr./Prof. Dr. Ext. Examiner (External Examiner)
Affiliation
Faculty, University

Title of Thesis:

Knowledge-Augmented Large Language Models for Arabic Post-OCR Error Correction

Key Words:
Arabic OCR; Post-OCR Correction; Large Language Models; Prompt Engineering; Retrieval-Augmented Generation; Qwen3; PATS-A01; Error Taxonomy

Summary:
This thesis investigates whether a lightweight open-source instruction-tuned large language model can correct errors produced by the Qaari Arabic OCR engine through prompt engineering alone, and whether injecting linguistic or retrieval-based knowledge into the prompt improves correction quality. Six isolated knowledge-augmentation strategies are evaluated on the PATS-A01 Akhbar typewritten dataset and compared against a zero-shot baseline. Results show that zero-shot correction reduces character error rate by 66% relative to the OCR baseline when accounting for catastrophic repetition artifacts, but increases error rate on clean text due to over-correction. All knowledge-augmentation strategies degrade performance relative to zero-shot, with RAG producing the worst result (CER 22.10%) and few-shot producing the least degradation (CER 10.09%). The dominant failure mode is over-correction: the model changes tokens that the OCR rendered correctly at a rate that exceeds errors fixed.

Disclaimer

I hereby declare that this thesis is my own original work and that no part of it has been submitted for a degree qualification at any other university or institute.

I further declare that I have appropriately acknowledged all sources used and have cited them in the references section.

Name: Engineer's Name Date:

Signature:

Dedication

You may include this section if you wish to dedicate your thesis to someone.

Acknowledgements

In this section, you may provide acknowledgements to those who gave you support and encouragement to complete your thesis. Acknowledgement of funding from local and international funding agencies must be clearly stated.

Table of Contents

Disclaimer **1**

Dedication **2**

Acknowledgements **3**

Table of Contents **4**

List of Tables **12**

List of Figures **14**

List of Symbols and Abbreviations **15**

List of Publications **17**

Abstract **18**

1 INTRODUCTION **20**

1.1 The Problem with Arabic OCR 1

1.2 The Open-Source vs. Closed-Source Gap 2

1.3 Post-OCR Correction as a Text-to-Text Task 3

1.4 Research Questions 4

1.5 Contributions 5

1.6 Thesis Organisation 6

2 BACKGROUND AND LITERATURE REVIEW **8**

2.1 The Arabic Language and Script 9

2.1.1 Linguistic Overview 9

2.1.2 The Arabic Script 10

2.1.3 Arabic Text in Computing 12

2.1.4 Challenges for Arabic NLP 12

2.2 Optical Character Recognition 13

2.2.1 What Is OCR? 13

2.2.2 OCR Architectures 14
2.2.3 Arabic-Specific OCR Challenges 15
2.2.4 Qaari: The OCR Engine Used in This Study 16
2.2.5 Evaluation Metrics for OCR 16
2.3 Large Language Models 17
2.3.1 From N-grams to Transformers 17
2.3.2 The Transformer Architecture 18
2.3.3 Pre-Training and Instruction Tuning 19
2.3.4 Qwen3-4B-Instruct-2507 20
2.3.5 Arabic LLMs and Arabic NLP with LLMs 20
2.4 Prompt Engineering 21
2.4.1 Zero-Shot Prompting 21
2.4.2 Few-Shot Prompting 22
2.4.3 Chain-of-Thought and Instruction Augmentation 23
2.4.4 Retrieval-Augmented Generation 23
2.4.5 Self-Reflective and Meta-Cognitive Prompting 24
2.5 Knowledge Sources Used in This Study 25
2.5.1 Confusion Matrix as a Knowledge Source 25
2.5.2 Arabic Orthographic Rules 25
2.5.3 The QALB Corpus 26
2.5.4 The OpenITI Corpus 26
2.5.5 CAMeL Tools 27
2.6 Related Work on Post-OCR Correction 28
2.6.1 Classical Approaches 28
2.6.2 Neural Sequence-to-Sequence Correction 28
2.6.3 LLM-Based Post-OCR Correction 29
2.6.4 Arabic-Specific OCR Correction 30
2.6.5 Positioning of This Work 31

3 SYSTEM DESIGN AND DATASETS 32
3.1 Overall Architecture 33
3.1.1 Three-Stage Pipeline 34
3.1.2 JSONL Record Format 35

3.1.3 Resume and Force-Rerun Behaviour 35
3.2 Datasets 36
3.2.1 PATS-A01 — Printed Arabic Text Set 36
3.2.1.1 Fonts and Coverage 36
3.2.1.2 Ground-Truth Encoding 37
3.2.1.3 Train/Validation Split 37
3.2.1.4 Scope of This Thesis 37
3.2.2 KHATT — Khatt Arabic Handwritten Text 38
3.2.3 Dataset Statistics 38
3.2.4 OCR Results Format 38
3.3 Preprocessing and Evaluation 39
3.3.1 Text Normalisation 39
3.3.2 CER and WER Computation 40
3.3.2.1 Definitions 40
3.3.2.2 Implementation 41
3.3.3 Error Taxonomy Construction 41
3.4 Knowledge Base Components 42
3.4.1 Confusion Matrix Loader 43
3.4.2 Rules Loader 43
3.4.3 QALB Loader and Filtering 44
3.4.4 OpenITI Corpus and RAG Index 45
3.4.4.1 Corpus Processing 46
3.4.4.2 FAISS Retrieval Index 46
3.4.5 CAMeL Tools Integration 46
3.4.5.1 MorphAnalyzer 47
3.4.5.2 WordValidator and the Revert Strategy 47
3.4.6 LLM Insights Loader 48
3.5 The LLM Correction Module 48
3.5.1 Class Hierarchy 48
3.5.2 Generation Configuration 49
3.5.3 Response Extraction 50
3.6 PromptBuilder 51

3.6.1 Prompt Design Philosophy 51
3.6.2 Prompt Variants 52
3.6.2.1 Zero-Shot Prompt (p2v2) 52
3.6.2.2 OCR-Aware Prompt (p3v1) 53
3.6.2.3 Rule-Augmented Prompt (p4av1) 53
3.6.2.4 Few-Shot Prompt (p4bv1) 53
3.6.2.5 Self-Reflective Prompt (p4dv1) 54
3.6.2.6 RAG Prompt (p5v1) 54
3.6.2.7 Combined Prompt (p6v1) 54

4 EXPERIMENTAL METHODOLOGY 55

4.1 Experimental Design Principles 55
4.1.1 The Isolation Principle 55
4.1.2 Phase 2 as the Hub Baseline 56
4.1.3 Reproducibility Measures 56
4.1.4 Why Not Test All Combinations First 57
4.2 Phase 1 — Baseline and Error Taxonomy 57
4.2.1 Hypothesis 57
4.2.2 Method 57
4.2.3 Output Files 58
4.3 Phase 2 — Zero-Shot LLM Correction 59
4.3.1 Hypothesis 59
4.3.2 Method 59
4.3.3 Prompt Design Rationale 59
4.3.4 Output Files 60
4.4 Phase 3 — OCR-Aware Prompting 60
4.4.1 Hypothesis 60
4.4.2 Method 61
4.4.3 Variables 61
4.4.4 Output Files 61
4.5 Phase 4A — Rule-Augmented Prompting 62
4.5.1 Hypothesis 62
4.5.2 Method 62

4.5.3 Variables 62
4.5.4 Output Files 63
4.6 Phase 4B — Few-Shot Examples 63
4.6.1 Hypothesis 63
4.6.2 Method 63
4.6.3 Transfer Gap Discussion 64
4.6.4 Output Files 64
4.7 Phase 4C — CAMeL Morphological Post-Processing 64
4.7.1 Hypothesis 64
4.7.2 Method 65
4.7.3 Revert Strategy Rationale 66
4.7.4 Output Files 66
4.8 Phase 4D — Self-Reflective Prompting 66
4.8.1 Hypothesis 66
4.8.2 Three-Stage Method 67
4.8.2.1 Stage 1: Training-Split Analysis (analyze-train mode) 67
4.8.2.2 Insights Derived from Training Data 67
4.8.2.3 Stage 2: Export (export mode) 68
4.8.2.4 Stage 3: Analyze (analyze mode) 69
4.8.3 Output Files 69
4.9 Phase 5 — Retrieval-Augmented Generation 70
4.9.1 Hypothesis 70
4.9.2 Method 70
4.9.2.1 Build Mode 70
4.9.2.2 Export Mode 70
4.9.2.3 Analyze Mode 71
4.9.3 Retrieval Quality 71
4.9.4 Output Files 72
4.10 Phase 6 — Combinations and Ablation 72
4.10.1 Purpose 72
4.10.2 Combinations Tested 72
4.10.3 Ablation Design 72

4.10.4 Statistical Testing 73
4.10.5 Output Files 74
4.11 Phase Summary 74

5 RESULTS AND ANALYSIS 75

5.1 Phase 1 Results — OCR Error Characterisation 75
5.1.1 Aggregate Error Rates 75
5.1.2 Error Type Distribution 76
5.1.3 Confusion Matrix Analysis 77
5.1.4 Morphological Validity Analysis 78
5.2 Phase 2 Results — Zero-Shot LLM Baseline 79
5.2.1 Aggregate Performance 79
5.2.2 Comparison to Phase 1 79
5.2.2.0.1 Relative to all-samples OCR (CER 17.01%). 79
5.2.2.0.2 Relative to normal-samples OCR (CER 1.37%). 80
5.2.3 Per-Sample Analysis 80
5.3 Phases 3–5 Results — Knowledge Augmentation Evaluation 81
5.3.1 Phase 3 — Confusion Matrix Injection 81
5.3.2 Phase 4A — Rule-Augmented Prompting 82
5.3.3 Phase 4B — Few-Shot Examples 83
5.3.4 Phase 4C — CAMeL Morphological Post-Processing 84
5.3.5 Phase 4D — Self-Reflective Prompting 85
5.3.6 Phase 5 — Retrieval-Augmented Generation 86
5.4 Phase 6 Results — Combinations and Ablation 88
5.5 Summary and Ranking 89

6 DISCUSSION AND CONCLUSION 91

6.1 Revisiting the Research Questions 91
6.2 The Over-Correction Problem 96
6.2.1 What Over-Correction Is 96
6.2.2 Why Over-Correction Happens 97
6.2.3 Why Augmentation Makes Over-Correction Worse 98
6.2.4 Connection to the Broader Literature 98

6.3 When Would Augmentation Help? 99
6.3.0.0.1 Higher baseline CER. 99
6.3.0.0.2 Domain-matched retrieval corpus. 100
6.3.0.0.3 OCR-native few-shot pairs. 100
6.3.0.0.4 Larger or fine-tuned models. 100
6.3.0.0.5 Confidence-based selective correction. . . . 100
6.4 Implications for System Design 101
6.4.0.0.1 For high-quality typewritten Arabic OCR: use zero-shot, do not augment. 101
6.4.0.0.2 Apply LLM correction selectively, not universally. 101
6.4.0.0.3 The LLM is a safety net for runaway artefacts. 101
6.4.0.0.4 Include CAMeL post-processing as a zero-cost defensive layer. 102
6.4.0.0.5 Do not deploy RAG unless the retrieval corpus is domain-matched. 102
6.5 Limitations 102
6.5.0.0.1 Single dataset. 102
6.5.0.0.2 Single OCR engine. 102
6.5.0.0.3 Single LLM. 103
6.5.0.0.4 No statistical significance testing on isolated phases. 103
6.5.0.0.5 Phase 6 not yet executed. 103
6.5.0.0.6 QALB transfer gap. 103
6.5.0.0.7 Classical corpus for RAG. 103
6.5.0.0.8 Diacritic stripping in evaluation. 104
6.6 Future Work 104
6.6.0.0.1 Evaluate on KHATT and all PATS-A01 fonts. 104
6.6.0.0.2 Generate OCR-native few-shot pairs for Phase 4B. 104
6.6.0.0.3 Build a domain-matched retrieval corpus for Phase 5. 104

6.6.0.0.4 Fine-tune Qwen3-4B for conservative OCR correction. 105
6.6.0.0.5 Test with larger models. 105
6.6.0.0.6 Study over-correction as a standalone research question. 105
6.6.0.0.7 Confidence-guided selective correction. . . 105
6.6.0.0.8 Extension to dialectal Arabic OCR. 106
6.7 Conclusion 106

References **109**

Appendix A FULL PROMPT TEMPLATES **112**

Appendix B PHASE 4D SELF-REFLECTIVE INSIGHTS **120**

Appendix C CONFIGURATION FILES **124**

Appendix D PATS-A01 ERROR EXAMPLES **129**

List of Tables

3.1 Fields in the inference JSONL files. All fields from the export stage are preserved through inference; the `correction` and status fields are appended by the inference script. 35

3.2 Dataset summary. "Evaluated" is the number of samples used in experiments reported in this thesis. "—" indicates that statistics depend on writer count and session structure not yet fully catalogued. 39

3.3 Error taxonomy used in Phase 1 baseline analysis. Categories are ordered from most specific (dot confusion) to most general (other substitution). 42

3.4 Arabic orthographic rule categories used in Phase 4A. Each category targets a specific phenomenon that both human writers and OCR engines commonly mishandle. 44

3.5 Generation parameters for `TransformersCorrector`. 50

3.6 Prompt variant registry. Each phase uses exactly one variant. All prompts are in Arabic. Full prompt text is given in Appendix A. . . 52

4.1 Phase 4D training-split analysis results for PATS-A01 Akhbar. The LLM is Qwen3-4B-Instruct-2507 under the zero-shot v2 prompt (Phase 2 setting). Fix rate = fraction of baseline errors of that type that the LLM corrected. Introduction rate = fraction of correct ground-truth tokens of that type that the LLM incorrectly changed. $n < 10$ entries are marked with † (too few samples for reliable estimates). 68

4.2 Phase 6 combination inventory. Each combination is identified by a unique ID. Components: C = confusion matrix, R = rules, E = examples (few-shot), S = self-reflective insights, G = RAG, M = CAMeL post-processing. 73

4.3 Summary of all experimental phases. "Comparison" is the phase each experiment is evaluated against. 74

5.1 Phase 1 OCR baseline metrics on PATS-A01 Akhbar (200 samples). 75

5.2 Phase 1 error taxonomy on PATS-A01 Akhbar (all 200 samples, 2,063 total character errors). 76
5.3 Character-level confusion pairs from Phase 1 (all 9 unique pairs; out of 61 total substitutions). 77
5.4 Phase 2 zero-shot LLM correction metrics on PATS-A01 Akhbar (200 samples). Phase 1 all-samples figures are shown for reference. 79
5.5 Main results: CER and WER for Phases 1–5 on PATS-A01 Akhbar (200 samples). Relative change vs. Phase 2 zero-shot baseline. . . . 81
5.6 Phase 4D self-reflective insights derived from the PATS-A01 Akhbar training split. Fix rate: fraction of OCR errors of this type corrected by the LLM. Introduction rate: fraction of correct tokens of this type incorrectly modified by the LLM. 85
5.7 Phase 6 combination designs. Results pending execution. 88
5.8 Final ranking of all executed experimental conditions by CER. Lower CER is better. Phase 6 results pending. 89
5.9 Summary of research question answers. 90

2.1 Per-type LLM performance statistics on PATS-A01 Akhbar training split (200 samples). Null entries indicate zero baseline instances. . . 123

4.1 Representative OCR error examples from Phase 1, PATS-A01 Akhbar training split. Context is a three-character window around the error. GT = ground truth; OCR = OCR output. 130

List of Figures

3.1 Three-stage pipeline architecture. Stages communicate via JSONL files on disk, enabling inference to run on any cloud GPU without requiring direct access to local data files. 34

3.2 LLM corrector class hierarchy. All pipeline code depends only on the `BaseLLMCorrector` interface; the concrete backend is selected at runtime by `get_corrector(config)`. 49

List of Symbols and Abbreviations

Symbol	Description
C	Number of character substitutions, deletions, and insertions (edit operations)
N	Total number of characters in the reference (ground-truth) text
W	Number of word substitutions, deletions, and insertions
M	Total number of words in the reference text
CER_i	Character error rate for sample i
n	Number of evaluation samples
$d(\cdot)$	Edit distance (Levenshtein distance) function
k	Number of retrieved passages in RAG (top-k)
α	Family-wise error rate for Bonferroni correction

Abbreviation	Description
CER	Character Error Rate
WER	Word Error Rate
LLM	Large Language Model
VLM	Vision–Language Model
OCR	Optical Character Recognition
RAG	Retrieval-Augmented Generation
MSA	Modern Standard Arabic
RLHF	Reinforcement Learning from Human Feedback
SFT	Supervised Fine-Tuning
NLP	Natural Language Processing
FAISS	Facebook AI Similarity Search

PATS	Printed Arabic Text Set
KHATT	Khatt Arabic Handwritten Text dataset
QALB	Qatar Arabic Language Bank
OpenITI	Open Islamicate Texts Initiative
CNN	Convolutional Neural Network
LSTM	Long Short-Term Memory network
CTC	Connectionist Temporal Classification
CoT	Chain-of-Thought (prompting)
GT	Ground Truth
std	Standard deviation
p95	95th percentile

List of Publications

1. [**Under preparation**] Knowledge-Augmented Large Language Models for Arabic Post-OCR Error Correction. *To be submitted to an IEEE conference on Arabic NLP or document analysis.*

Abstract

Arabic optical character recognition (OCR) engines produce systematic errors rooted in the visual properties of the Arabic script: dot-group character confusions, morphologically valid but contextually wrong substitutions, and occasional catastrophic repetition artefacts. Open-source engines such as Qaari fall short of closed-source alternatives, motivating the question of whether post-OCR correction via a large language model (LLM) can close this gap without modifying the recognition engine.

This thesis presents a systematic evaluation of prompt-engineering strategies for Arabic post-OCR correction using Qwen3-4B-Instruct-2507, an open-source 4-billion-parameter instruction-tuned model. Six knowledge augmentation conditions are evaluated in isolation on 200 samples of the PATS-A01 Akhbar typewritten corpus, each changing exactly one variable relative to a zero-shot baseline: (1) character confusion matrix injection (Phase 3); (2) Arabic orthographic rule augmentation (Phase 4A); (3) few-shot examples from the QALB corpus (Phase 4B); (4) CAMeL Tools morphological post-processing (Phase 4C); (5) self-reflective prompting from training-split error statistics (Phase 4D); and (6) retrieval-augmented generation with the OpenITI classical corpus (Phase 5). A combination and ablation study (Phase 6) tests twelve pre-selected multi-component systems.

The Qaari baseline achieves 1.37% character error rate (CER) on normal samples and 17.01% overall, with the discrepancy caused by a repetition artefact in 0.5% of samples. Zero-shot LLM correction (Phase 2) reduces aggregate CER to 5.77% (66% relative reduction) by collapsing repetition artefacts, but raises CER on normal samples from 1.37% to 5.77%, revealing a systematic over-correction problem: the model modifies correct tokens at a 25.06% error introduction rate. Every knowledge augmentation strategy tested degrades performance relative to the zero-shot baseline, with degradations ranging from 75% (Phase 4B few-shot) to 283% (Phase 5 RAG). CAMeL post-processing has no effect, as all OCR errors and LLM corrections are morphologically valid Arabic forms and the revert condition is never triggered.

The central finding is that, on high-quality typewritten Arabic input, more prompt information consistently worsens correction because each additional knowledge

element activates the model's intervention tendency, increasing the rate at which correct tokens are corrupted. The simplest strategy — zero-shot correction applied selectively to high-error samples — is both the most effective and the most practical recommendation. Future work should evaluate on handwritten text (KHATT), replace QALB with OCR-native few-shot pairs, and build domain-matched retrieval corpora to test whether these findings generalise to higher-error-rate and domain-aligned settings.

Chapter 1

Introduction

1.1 The Problem with Arabic OCR

The digitisation of Arabic text is a prerequisite for a broad class of downstream applications: full-text search over historical archives, neural machine translation of Arabic news and literature, text-to-speech systems for visually impaired users, and information extraction from scanned government documents. The technology that converts images of text into machine-readable character sequences — optical character recognition (OCR) — is the entry point for all of these applications. When OCR fails, the failure propagates: a misread character changes word meaning, a sequence of recognition errors degrades translation quality, and a corrupted document image cannot be retrieved by keyword search.

Arabic presents challenges that English OCR does not face. The Arabic script is cursive: letters connect within words and their shapes change depending on their position in the word (isolated, initial, medial, or final). There are no clear boundaries between characters within a word, so segmentation and recognition are tightly coupled — a segmentation error directly produces a recognition error. Many Arabic letters share the same base skeleton and differ only in the number and placement of small dots: the pair ب/ي (ba/ya) differs only in whether the dot is below or above the baseline; ف/ق (fa/qaf) differs only in having one or two dots above. Ink irregularities, low scan resolution, or font variation can shift a dot from its correct position and cause the OCR engine to output the wrong letter.

Short vowels (diacritics, or *tashkeel*) are generally optional in modern printed Arabic and are absent from most typewritten text, yet their presence or absence changes word meaning and complicates matching between OCR output and ground

truth. Classical and modern standard Arabic coexist with several regional dialects, each with partially distinct orthographies. Ligatures — special glyphs for common letter combinations — further increase the visual complexity that OCR engines must resolve.

The consequence of these challenges is that open-source Arabic OCR engines produce recognisable error patterns. Character-level confusions between visually similar letter pairs, word-level segmentation failures, and occasional catastrophic output loops inflate the character error rate (CER) in ways that downstream applications cannot silently absorb. A single confused character in a word can change its meaning entirely; a repeated output loop corrupts an entire document page.

1.2 The Open-Source vs. Closed-Source Gap

The landscape of Arabic OCR is divided between commercial closed-source systems and open-source alternatives. Closed-source vision–language models (VLMs) — including GPT-4V [1], Gemini, and similar systems — handle Arabic document recognition with high accuracy because they were trained on massive multimodal corpora that include Arabic text in diverse scripts, fonts, and document layouts. These systems are not publicly auditable, cannot be deployed in offline or sensitive environments, and charge per-query fees that are prohibitive for large archival digitisation projects.

Open-source OCR engines such as Qaari[1] offer accessibility, reproducibility, and deployability without network dependencies. However, they fall behind closed-source alternatives on error rate, particularly on difficult fonts, handwritten text, and documents with low scan quality. The gap is meaningful: a 1–2% CER difference translates to dozens of errors per page in a long document.

The question motivating this thesis is whether this gap can be narrowed *without changing the OCR engine itself.* Replacing an open-source OCR engine with a closed-source one is not always feasible. But adding a post-processing correction stage — a language model that receives the OCR output as text and corrects it before

[1]Qaari is an open-source Arabic OCR system developed for modern printed Arabic text. It is the primary OCR engine evaluated in this thesis.

downstream use — is a modular intervention that leaves the OCR pipeline intact. If a language model can reliably correct the systematic errors that open-source Arabic OCR engines make, the effective accuracy of the system would improve without requiring a new recognition model.

1.3 Post-OCR Correction as a Text-to-Text Task

Post-OCR correction treats the OCR output as noisy text and uses a language model to produce a denoised version. The formulation is simple: given a sequence of characters or words output by the OCR engine, produce the most likely corresponding sequence in the original document. This is a text-to-text task — no images are required after the OCR stage — and can be approached with any sequence-to-sequence model.

The decoupling of the vision stage (OCR) from the language stage (correction) has several practical advantages. It is engine-agnostic: the same correction system can be applied to output from any OCR engine. It is retroactive: it can improve existing digitised archives without re-scanning or re-running OCR. It is modular: improvements to the OCR engine and improvements to the corrector can be developed independently. And it is accessible: language models capable of Arabic text processing are available as open-source checkpoints that can be run without proprietary APIs.

The rise of large language models (LLMs) trained on massive multilingual corpora opens a new avenue for post-OCR correction. Unlike classical correction approaches that require a specialised language model trained on OCR error pairs [2], a general-purpose instruction-tuned LLM can be directed to perform correction tasks through a natural-language prompt. This *prompt engineering* approach requires no task-specific training, no labelled correction pairs, and no model fine-tuning. The question is whether such a model — given only a prompt and the OCR text — can correct Arabic OCR errors effectively, and whether injecting additional knowledge into the prompt (error patterns, orthographic rules, examples, retrieved context) can further improve performance.

1.4 Research Questions

This thesis investigates the following research questions.

RQ1: What are the characteristics of errors produced by Qaari on Arabic typewritten text? What error types dominate, and how severe is the overall degradation?

RQ2: Can a zero-shot instruction-tuned LLM correct Arabic OCR errors without any task-specific guidance?

RQ3: Does injecting OCR-specific error knowledge (character confusion matrix) into the prompt improve correction beyond zero-shot?

RQ4: Does injecting explicit Arabic orthographic rules into the prompt improve correction?

RQ5: Does providing error-correction examples from the QALB corpus as few-shot demonstrations improve correction?

RQ6: Does morphological post-processing via CAMeL Tools — reverting LLM-introduced morphological invalids to the OCR original — improve the quality of corrected text?

RQ7: Does self-reflective prompting — injecting statistical summaries of the model's own correction failures, derived from a training split — improve correction on a held-out evaluation set?

RQ8: Does retrieval-augmented generation (RAG) grounded in the OpenITI classical Arabic corpus improve correction?

RQ9 (overarching): Which combination of knowledge augmentation strategies produces the best correction quality, and what does each component individually contribute in ablation?

RQ1 is answered in Chapter 5.5, Section 5.1. RQ2–RQ8 are each answered by a corresponding isolated experiment (Phases 2–5). RQ9 is answered by the combination and ablation study (Phase 6), results for which are pending. Research questions RQ2–RQ9 all take Phase 2 zero-shot correction as their reference point for comparison.

1.5 Contributions

This thesis makes the following contributions.

1. **A systematic evaluation framework for Arabic post-OCR correction.** Six isolated knowledge augmentation conditions are evaluated on a common benchmark (PATS-A01 Akhbar, 200 samples, Qwen3-4B), each changing exactly one variable relative to the zero-shot baseline. This isolation design enables direct attribution of observed CER changes to individual knowledge sources.

2. **A three-stage export–infer–analyse pipeline.** The experimental infrastructure separates local data preparation from GPU inference from result analysis. Inference is executed on Kaggle (free GPU quota) via a unified `scripts/infer.py` script that handles resumption, HuggingFace synchronisation, and prompt versioning. This design enables full reproducibility without a local GPU.

3. **A novel self-reflective prompting strategy.** Phase 4D derives correction guidance from the model's own training-split error statistics — quantifying fix rates and error introduction rates per error type, converting them into Arabic-language diagnostic sentences, and injecting them into the system prompt for validation-split inference. This strategy is, to our knowledge, the first application of self-diagnostic statistics as prompt augmentation for Arabic OCR correction.

4. **An empirical finding on the over-correction problem.** Zero-shot LLM correction is the best-performing strategy for high-quality typewritten Arabic OCR, outperforming all six knowledge-augmented variants. The primary failure mode of all augmented conditions is over-correction: a 25.06% error introduction rate means the LLM corrupts 1 in 4 previously correct tokens. Every augmentation element tested increases this rate rather than reducing it.

5. **An error taxonomy for Qaari on PATS-A01 Akhbar.** Phase 1 produces a character-level confusion matrix, an eight-category error taxonomy (dot

confusion, alef maqsura, similar shape, hamza, taa marbuta, insertion, deletion, other substitution), and a CAMeL Tools morphological analysis confirming that 100% of Qaari errors on this corpus are valid-but-wrong substitutions, not morphologically invalid non-words.

6. **Open-source code and reproducible configuration.** All pipelines, prompt templates, knowledge base loaders, and evaluation scripts are publicly available under a permissive licence. Every experimental result is reproducible from the repository using the provided configuration files and inference notebooks.

1.6 Thesis Organisation

The remainder of this thesis is organised as follows.

Chapter 2.6.5 — Background and Literature Review builds the necessary foundations. It covers the Arabic language and script, the evolution of OCR from template matching through deep learning to vision–language models, the development of large language models and instruction tuning, prompt engineering strategies (zero-shot, few-shot, chain-of-thought, RAG, self-reflective), and a survey of related work on post-OCR correction and Arabic NLP. A reader familiar with any of these topics may skip the corresponding subsection.

Chapter 3.6.2.7 — System Design and Datasets describes the experimental infrastructure. It introduces the PATS-A01 and KHATT datasets, explains the three-stage export–infer–analyse pipeline, defines the text normalisation and CER/WER evaluation protocol, and documents each knowledge base component (confusion matrix loader, rules loader, QALB loader, OpenITI/FAISS retriever, CAMeL Tools integration, and LLM insights loader). The LLM correction module and PromptBuilder class hierarchy are also described here.

Chapter 4.11 — Experimental Methodology describes each experimental phase as a self-contained experiment with a stated hypothesis, method, and expected output. Phases 1 through 6 are covered, with particular attention to the isolation principle that ensures each augmentation condition can be attributed to a single knowledge source.

Chapter 5.5 — Results and Analysis reports all numerical results for Phases

1–5 and the Phase 6 design. Every result is stated in a table and interpreted in prose. The chapter answers RQ1–RQ8 and characterises the over-correction problem quantitatively.

Chapter 6.7 — Discussion and Conclusion interprets the findings, develops the over-correction problem as a unified theoretical argument, discusses the conditions under which the results would likely reverse, states practical system design recommendations, enumerates limitations, and charts future research directions. A reader primarily interested in conclusions and implications may begin with this chapter.

The appendices provide supporting material: Appendix A contains the full prompt templates for all phases in both Arabic and English; Appendix B contains the Phase 4D Arabic-language diagnostic text generated from training-split analysis; Appendix C contains the relevant sections of the configuration file; and Appendix D contains a representative table of OCR error examples from Phase 1.

Chapter 2

Background and Literature Review

This chapter builds the background necessary to follow the rest of the thesis. A reader unfamiliar with Arabic script, OCR technology, large language models, or prompt engineering should be able to acquire the required context here. Section 2.1 covers the Arabic language and script. Section 2.2 introduces optical character recognition and its Arabic-specific challenges. Section 2.3 surveys large language models from early neural approaches through the modern transformer era. Section 2.4 covers prompt engineering methods relevant to this study. Section 2.5 describes the external knowledge sources used. Section 2.6 reviews related work on post-OCR correction and positions this thesis within that literature.

2.1 The Arabic Language and Script

2.1.1 Linguistic Overview

Arabic is a Semitic language and the liturgical language of Islam, spoken by over 300 million people as a first language across the Arab world and understood by a much larger community as a second or religious language [3]. Its most distinctive grammatical feature is *root-and-pattern morphology*: the vast majority of Arabic words are derived from a three-consonant (trilateral) root, and different word patterns are created by interleaving the root consonants with fixed vowel-and-consonant patterns. For example, the root *k-t-b* (relating to writing) yields: *kataba* (he wrote), *kitaab* (book), *kaatib* (writer), *maktaba* (library), *maktub* (written/letter), and dozens of other morphologically related forms.

This morphological richness creates a large and sparse vocabulary for text processing tools. Where English uses analytic constructions ("he will have been writing"), Arabic encodes tense, aspect, voice, gender, number, and definiteness through bound morphology, generating many distinct surface forms for each lemma. A vocabulary estimated at several hundred thousand distinct word forms makes statistical NLP methods data-hungry and challenges neural models trained on smaller corpora.

Modern Standard Arabic (MSA) is the standardised written form used in formal contexts across the Arab world: newspapers, government documents, academic publications, and formal broadcasts. It descends from Classical Arabic (the language of the Quran) with some modernisation of vocabulary. Alongside MSA, a spectrum of spoken dialects exists (Egyptian, Gulf, Levantine, Maghrebi, etc.) that differ substantially from MSA and from each other in phonology, morphology, and lexicon. This study works with MSA throughout, as PATS-A01 documents are printed in MSA and the QALB and OpenITI corpora also contain MSA text.

2.1.2 The Arabic Script

Arabic is written with a 28-letter abjad (consonantal alphabet) that runs from right to left. Several properties of the script are critical for understanding OCR failure modes.

Positional forms. Most letters have four shapes depending on their position within a word: isolated, initial, medial, and final. The letter *ain* (ع), for instance, appears as ع (isolated), عـ (initial), ـعـ (medial), and ـع (final). Since letters are connected into words by ligatures (joined strokes), an OCR engine must recognise not just a character in isolation but a glyph in context, making segmentation errors compound recognition errors.

Diacritics (tashkeel / harakat). Short vowels in Arabic are written as small marks placed above or below consonant letters. The primary marks are: fatha (short /a/, above), kasra (short /i/, below), damma (short /u/, above), sukun (consonant

with no vowel, above), and shadda (gemination, above). Tanwin (nunation) suffixes – tanwin al-fath, tanwin al-kasr, tanwin al-damm – mark indefinite nouns and consist of doubled fatha, kasra, or damma marks. In most modern Arabic text, including newspapers, diacritics are *omitted* from printed text and readers infer the correct vowelisation from context. When present, diacritics are critical for disambiguation: the same consonant sequence can represent multiple words depending on vowelisation.

Hamza. The glottal stop (hamza) has a complex orthography. In some positions it appears alone (ء); in others it appears on a carrier letter: alef (أ above, إ below), waw (ؤ), or ya without dots (ئ). The choice of carrier follows rules based on the surrounding vowels, and even educated native speakers sometimes apply these rules inconsistently. Hamza rules are a major source of spelling errors in human writing and a significant challenge for OCR.

Taa marbuta. The feminine ending ة (taa marbuta) is pronounced /at/ in context and /a/ in pause. It is written with the same base shape as the letter ha (ه) but with two dots above. The visual similarity between ة and ه is a direct source of OCR errors whenever the ink quality, resolution, or font rendering is imperfect.

Alef maqsura. The letter ى (alef maqsura, literally "abbreviated alef") appears at the end of certain words to write the long vowel /aa/. Its shape is identical to ya (ي) but without the two dots below. OCR engines regularly confuse ى and ي, and the distinction is orthographically important because the two letters signal different word identities.

Dot-group characters. Six groups of letters share the same base skeletal shape and differ only in the count or position of diacritical dots above or below: ب / ت / ث (1/2/3 dots above), ج / ح / خ (1 dot below / none / 1 dot above), ف / ق (1 dot above / 2 dots above), ن / ي (1 dot above / 2 dots below), and others. Dot placement errors – caused by ink smearing, low resolution, or font rendering artefacts – are the most frequent single-character OCR confusion in Arabic typewritten text.

2.1.3 Arabic Text in Computing

Arabic text in digital systems is encoded in Unicode. The primary Arabic Unicode block spans U+0600 to U+06FF and covers the standard 28 letters, their positional variants, diacritical marks, punctuation, and numerals. The diacritical marks (tashkeel) occupy code points U+064B through U+065F for the standard marks, with additional marks in the ranges U+0610–U+061A and U+0670. These code points can be removed programmatically to strip diacritics without affecting the consonantal skeleton.

Right-to-left rendering requires bidirectional text support. The Unicode Bidirectional Algorithm (UBA) governs how mixed LTR/RTL content is laid out. Documents containing both Arabic and numerals (which are LTR) or Arabic and English words exhibit complex rendering behaviour that can cause misalignment between stored byte order and visual display order. OCR engines and NLP tools must handle this correctly to avoid misinterpreting character sequences.

Legacy encodings such as Windows Code Page 1256 (CP-1256), ISO-8859-6, and the ASMO-708 standard predate Unicode and are still found in digitised Arabic document collections. The PATS-A01 ground-truth files, for example, are stored in CP-1256. Correct decoding requires explicit encoding specification; reading an Arabic CP-1256 file as UTF-8 produces garbled output. All text in this study is converted to UTF-8 on load.

2.1.4 Challenges for Arabic NLP

Tokenisation. Arabic word boundaries are clear (words are separated by spaces), but morpheme boundaries within words are not. Arabic clitics – conjunctions, prepositions, definite articles, pronominal suffixes – are written attached to the host word without any separator. Tokenising the sentence into morphological units requires a separate segmentation step beyond whitespace splitting.

Vocabulary sparsity. The large number of inflectional and derivational forms

per lemma means that any fixed-vocabulary system encounters out-of-vocabulary tokens frequently unless the vocabulary is very large. Subword tokenisation approaches (byte-pair encoding, WordPiece) partially address this but fragment morphologically complex words into unintuitive subword units.

Dialectal variation. Egyptian, Gulf, Levantine, and Maghrebi dialects differ from MSA and from each other in phonology, morphology, and lexicon. A model trained on MSA may perform poorly on dialectal text. This study focuses on MSA text throughout.

Code-switching. Modern Arabic digital text frequently mixes Arabic script with embedded English words, numbers, emoticons, and URLs. Handling such mixed-script content robustly requires robust character-level processing.

2.2 Optical Character Recognition

2.2.1 What Is OCR?

Optical Character Recognition (OCR) is the process of converting images of text into machine-readable character sequences. The classical OCR pipeline consists of four stages:

1. **Preprocessing.** The scanned or photographed document image is prepared for analysis: binarisation (converting to black-and-white), deskewing (correcting rotational distortion), denoising (removing salt-and-pepper and background noise), and normalisation of brightness and contrast.

2. **Layout analysis.** The preprocessed image is segmented into regions: page borders, columns, paragraphs, text lines, and (in character-segmenting systems) individual character images. For Arabic, line detection must handle the right-to-left reading order and the slanted or curved baselines that appear in handwritten text.

3. **Character recognition.** The character recognition module maps each segmented glyph (or a sequence of glyphs, in modern end-to-end systems) to a Unicode character. This is the core of OCR and the component that has changed most dramatically with deep learning.

4. **Post-processing.** A language model or lexicon is consulted to resolve ambiguous recognitions and correct plausible errors. In classical systems this was a rule-based or n-gram language model step. In modern systems it is sometimes omitted because the recognition model is end-to-end and incorporates linguistic knowledge implicitly.

2.2.2 OCR Architectures

The OCR field has passed through three distinct architectural eras:

Template matching (pre-2000). Early systems compared each segmented character image against a library of stored templates and selected the nearest match by pixel distance or correlation [4]. Template matching is brittle with respect to font variation, noise, and degradation, and requires re-training the template library for each new font.

Hidden Markov Models (1990s–2010s). HMM-based systems model the sequence of observations (image features along a text line) as a probabilistic generative process. They can handle cursive scripts without explicit character segmentation, which made them the dominant approach for Arabic OCR through the 2000s. A character language model is typically combined with the HMM in a noisy channel framework to improve recognition.

Deep learning era (2014–present). Convolutional neural networks replaced hand-crafted feature extraction. The combination of CNN feature extraction, Bidirectional LSTM sequence modelling, and Connectionist Temporal Classification (CTC) decoding [5] became the dominant paradigm for text-line recognition. Tesseract v4, the open-source OCR engine on which Qaari is based, adopted this CNN-LSTM-CTC architecture [6].

Transformer-based OCR (2021–present). Vision-language models such as TrOCR [7] and Donut [8] treat OCR as a sequence-to-sequence generation task: a vision encoder encodes the document image, and a text decoder generates the recognised text token by token. These models are pre-trained on large paired image-text datasets and achieve state-of-the-art performance on printed and even handwritten text. However, they require substantial compute and training data, which limits their availability for low-resource languages and scripts.

2.2.3 Arabic-Specific OCR Challenges

The properties of the Arabic script described in Section 2.1 translate directly into OCR failure modes.

Segmentation errors. Because Arabic letters are connected within a word, a segmentation error (incorrectly splitting a ligature or incorrectly merging two letters) cascades into recognition errors for all characters involved. English OCR, which deals mostly with separated or lightly connected Latin characters, does not face this problem to the same degree.

Dot placement ambiguity. The fine dots above and below base letter shapes carry discriminative information but are small relative to the letter body. Low image resolution, ink bleed, or scan artefacts can render dots illegible, and the OCR engine must then guess between all members of the relevant dot group. This is the most frequent source of character-level errors in PATS-A01.

Ligature handling. Certain letter combinations in Arabic merge into single ligature glyphs with a shape that may not visually resemble the component letters in isolation. Common examples include lam-alef (ﻻ), which is always written as a single glyph. OCR engines trained on character-level templates may not include all ligature shapes, leading to substitution or insertion errors.

Diacritic sensitivity. OCR engines vary in whether they attempt to recognise diacritics. Many ignore them entirely, which is appropriate for most modern Arabic text but problematic for classical or religious text where diacritics are always present.

Others attempt recognition but produce errors when diacritics are present in unusual positions or rendered in non-standard fonts.

Baseline variability. In handwritten text, words do not lie on a flat baseline. Variable baseline trajectories challenge text-line segmentation and make aligned feature extraction more difficult.

2.2.4 Qaari: The OCR Engine Used in This Study

Qaari is an open-source Arabic OCR engine based on Tesseract v4 with Arabic LSTM models trained specifically for Arabic printed text. It represents the class of freely available, community-maintained OCR engines that are accessible to researchers and practitioners who cannot afford commercial OCR solutions.

In the experiments of this thesis, Qaari performs well on clean PATS-A01 Akhbar text: the character error rate on normal samples is 1.37% (Section 5.5). However, Qaari exhibits one notable failure mode: a *repetition artifact*. When processing certain document layouts, Qaari enters a loop in which it repeatedly outputs the same short text span, producing a hypothesis that is hundreds of times longer than the ground truth. This artifact is rare (observed in 1 out of 200 samples, 0.5%) but when it occurs it produces an extremely high per-sample CER that dominates the aggregate. The artifact is a real-world behaviour of the OCR engine and is included in all evaluations to preserve ecological validity.

2.2.5 Evaluation Metrics for OCR

The two standard metrics for evaluating OCR and post-OCR correction are Character Error Rate (CER) and Word Error Rate (WER), both defined in Section 3.3 (Chapter 3). Here we comment on the choice of CER as the primary metric and on the diacritics stripping decision.

Why CER over WER. Arabic morphology creates a large space of valid word forms; any character-level error produces a word-level error. WER is therefore

always higher than CER for Arabic text and is more sensitive to the tokenisation convention used. CER captures the actual number of character edits needed, which is more directly interpretable as a measure of recognition accuracy at the signal level. CER is also the standard primary metric in post-OCR correction shared tasks [9].

Diacritics stripping. Evaluating with diacritics would penalise a corrector for failing to reproduce or adding diacritics that are stylistically optional. Since neither Qaari nor the LLM corrector is designed to hallucinate or remove diacritics that are part of the ground truth, evaluating without diacritics provides a cleaner measure of lexical correction quality. Both variants are computed and stored; the diacritic-stripped metric is reported as primary throughout this thesis.

2.3 Large Language Models

2.3.1 From N-grams to Transformers

Statistical language modelling has a long history in NLP. N-gram language models assign a probability to a sequence of words by decomposing the joint probability into a product of conditional probabilities of each word given the preceding $n-1$ words. They were the dominant approach for language modelling and speech recognition through the 1990s and 2000s. Their limitations – sparse counts for long contexts and inability to generalise across syntactically similar contexts – motivated neural approaches.

Bengio et al. [10] introduced the neural probabilistic language model, which represents words as dense real-valued vectors (embeddings) and uses a feedforward network to predict the next word given a fixed-width context window. This model demonstrated that neural approaches could outperform n-gram models by generalising across similar word contexts.

Recurrent Neural Networks (RNNs) extended this to variable-length contexts. Long Short-Term Memory (LSTM) networks [11] addressed the vanishing gradient problem of vanilla RNNs with gated memory cells, enabling effective modelling

of long-range dependencies. LSTM-based language models and encoder-decoder sequence-to-sequence models dominated NLP in the early 2010s.

The transformer architecture [12], introduced in 2017, replaced recurrence with self-attention mechanisms. Transformers train orders of magnitude faster than LSTMs on parallel hardware and scale more effectively with data and model size. All modern large language models are transformer-based.

2.3.2 The Transformer Architecture

The key innovation of the transformer is the *self-attention mechanism*, which allows every token in a sequence to directly attend to every other token, regardless of their distance. Given a sequence of token embeddings represented as a matrix $X \in \mathbb{R}^{nd}$, the scaled dot-product attention is computed as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V \tag{2.1}$$

where $Q = XW^Q$, $K = XW^K$, $V = XW^V$ are learned linear projections of the input into query, key, and value spaces, and d_k is the key dimension used for scaling. The softmax produces an attention weight distribution over all positions, and the output is a weighted sum of value vectors.

Multi-head attention runs h independent attention operations in parallel, each with its own projection matrices, and concatenates their outputs:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W^O \tag{2.2}$$

This allows the model to attend to information from different representational subspaces simultaneously.

Positional encoding is added to the input embeddings to provide information about token order, since the attention operation is otherwise position-invariant. Original transformers used sinusoidal positional encodings; modern LLMs often use rotary positional embeddings (RoPE) or similar learned variants.

A transformer block consists of a multi-head self-attention sublayer followed by a position-wise feedforward network, with residual connections and layer normalisation around each sublayer. Modern LLMs are decoder-only transformers: they use masked (causal) self-attention so that each token can only attend to tokens at or before its position, enabling autoregressive generation.

2.3.3 Pre-Training and Instruction Tuning

Modern LLMs are trained in two stages.

Pre-training. The model is trained to predict the next token given all preceding tokens, on a massive corpus of text scraped from the internet, books, code, and other sources. Pre-training instils broad language competence, world knowledge, and the ability to reason across many domains, all from the statistical structure of the training corpus. GPT-3 [1] demonstrated that scaling pre-training to 175 billion parameters enables dramatic improvements in few-shot performance across NLP tasks.

Instruction tuning and RLHF. A pre-trained model produces likely continuations of text; it does not reliably follow explicit instructions. Instruction tuning [13] fine-tunes the model on a curated set of instruction-response pairs, teaching it to interpret instructions as tasks and produce helpful, structured outputs. Reinforcement Learning from Human Feedback (RLHF) further aligns the model's outputs with human preferences by training a reward model on human comparisons and optimising the language model against it using policy gradient methods. The result is an instruction-tuned model that follows natural language instructions reliably across a wide range of tasks without requiring task-specific fine-tuning.

Instruction-tuned models are the right choice for post-OCR correction because they can follow a correction instruction directly. Unlike pre-trained base models, which require careful prompt engineering to elicit a specific output format, instruction-tuned models respond predictably to system messages of the form "correct this text and return only the corrected version."

2.3.4 Qwen3-4B-Instruct-2507

Qwen3-4B-Instruct-2507 is the primary LLM used in this study. It belongs to the Qwen3 family released by Alibaba's Tongyi team [14]. Key characteristics relevant to this study are as follows.

Scale and architecture. Qwen3-4B has approximately 4 billion parameters and is a decoder-only transformer. At this scale it fits comfortably in the 16 GB VRAM of a Kaggle T4 GPU in `float16` precision, making it a practical choice for cloud GPU inference without quantization.

Multilingual capability and Arabic support. Qwen3 was pre-trained on a large multilingual corpus covering over 100 languages, with substantial Arabic representation. This enables the model to read, generate, and reason in Arabic without any Arabic-specific fine-tuning.

Instruction following. The `-Instruct` variant is trained with supervised fine-tuning and preference alignment to follow instructions reliably. This is the essential property for post-OCR correction, where the model must interpret a system prompt and produce a structured text output.

Thinking mode. Qwen3 models include an optional chain-of-thought reasoning mode controlled by the `enable_thinking` parameter in `apply_chat_template()`. When enabled, the model generates an extended internal reasoning trace enclosed in `<think>...</think>` tokens before producing its final answer. This study disables thinking mode (`enable_thinking=False`) for two reasons: first, post-OCR correction is a direct transformation task that does not benefit from extended chain-of-thought reasoning; second, keeping thinking mode enabled adds latency and token cost, and the thinking trace contents sometimes contaminate the extracted correction.

2.3.5 Arabic LLMs and Arabic NLP with LLMs

The Arabic NLP community has produced several dedicated pre-trained models.

AraBERT [15] is an encoder-only BERT model pre-trained on a large corpus of Arabic news and Wikipedia text. It achieves strong results on Arabic classification and understanding tasks and established that pre-training specifically on Arabic outperforms multilingual BERT on Arabic benchmarks at comparable model size.

CAMeLBERT [16] extends this line of work with models pre-trained on MSA, dialectal Arabic, and mixed-domain corpora, providing tools tailored to different Arabic registers.

AraGPT2 [17] and **AraT5** [18] bring generative and text-to-text capabilities to Arabic, enabling Arabic language generation and conditional text transformation.

These Arabic-specific models, while valuable for their target tasks, operate at scales (hundreds of millions of parameters) far below current instruction-tuned LLMs. More recent results have shown that large multilingual instruction-tuned models such as GPT-4, Llama-3, and Qwen3 outperform Arabic-specific models on most Arabic NLP benchmarks, primarily because the scale advantage outweighs the benefit of Arabic-only pre-training. Qwen3-4B was chosen for this study as the most capable openly accessible model at a size deployable on free cloud GPU tiers.

2.4 Prompt Engineering

2.4.1 Zero-Shot Prompting

In zero-shot prompting, the model receives an instruction and an input without any demonstration examples [1, 19]. Performance depends entirely on the clarity of the instruction and the model's pre-training coverage of the task. For post-OCR correction, the zero-shot instruction must communicate both the task (correct errors in the text) and the constraint (do not change what is correct).

A key design decision is the degree of *aggressiveness* in the instruction. An aggressive prompt ("correct all errors in this text") primes the model to intervene on every uncertainty, producing rewrites of already-correct tokens. A conservative

prompt ("return the text unchanged if it is correct; fix only clear errors") primes the model to leave the text alone unless it is confident of an error. For a task where the vast majority of tokens in the input are already correct (OCR baseline CER of 1.37% on normal samples), conservative prompting is the rational choice. The design and evaluation of this choice is a central finding of Chapter 5.

2.4.2 Few-Shot Prompting

Few-shot prompting, introduced in GPT-3 [1], provides the model with a small number of input-output demonstration pairs before the actual query. This enables *in-context learning*: the model infers the task format and the expected transformation from the examples without any weight update.

Several practical decisions govern few-shot performance:

- **Number of examples.** More examples generally improve performance up to the point where they saturate the context window. This study uses 5 examples, balancing context length against representativeness.

- **Example selection.** Random selection may draw all examples from the same error type. Diversity-maximising selection ensures that the examples span multiple error categories, giving the model a broader template for what corrections look like.

- **Source of examples.** Phase 4B uses QALB corpus pairs. QALB annotations cover human typing errors (omitted diacritics, wrong hamza form, case-agreement errors) that have a partial but imperfect overlap with OCR errors (visual confusion between similar-shape characters). This *transfer gap* – the mismatch between human-error and OCR-error distributions – is a potential explanation for the performance outcome discussed in Chapter 5.

2.4.3 Chain-of-Thought and Instruction Augmentation

Chain-of-thought (CoT) prompting [20] instructs the model to generate intermediate reasoning steps before producing the final answer. CoT has been shown to improve performance on multi-step reasoning tasks such as arithmetic, symbolic manipulation, and commonsense inference.

Chain-of-thought is not used in this study. Post-OCR correction is a one-pass text-to-text transformation that does not require multi-step reasoning: for each position in the input, the model either copies the character or replaces it. There is no decomposition into sub-problems that would benefit from intermediate reasoning. Furthermore, enabling chain-of-thought in Qwen3 (via the thinking mode) adds substantial latency, unpredictable output length, and the risk of the reasoning trace contaminating the extracted correction. The conservative instruction in the system prompt already encodes the key cognitive constraint ("prefer no change unless certain") without requiring explicit reasoning steps.

2.4.4 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) [21] combines a parametric language model with a non-parametric retrieval component. At inference time, a retrieval system queries a large external corpus with an embedding of the input, retrieves the most similar passages, and provides these as additional context in the prompt. This allows the model to ground its outputs in specific textual evidence rather than relying solely on knowledge encoded in its weights.

The RAG pipeline consists of the following components:

1. **Corpus indexing.** The external corpus is split into sentences or passages, each embedded by a dense encoder into a fixed-dimensional vector. Embeddings are stored in a vector index that supports efficient nearest-neighbour search.

2. **Query encoding.** The input text is embedded using the same encoder.

3. **Retrieval.** The index is searched for the k corpus items with embeddings most similar to the query, typically by inner product or cosine similarity. This study uses the FAISS [22] library with a flat inner-product index.

4. **Prompt augmentation.** The retrieved passages are formatted and prepended to the system prompt as reference context.

This study uses the `paraphrase-multilingual-MiniLM-L12-v2` model [23] for both corpus and query encoding. This is a compact 12-layer multilingual sentence encoder with strong Arabic support, suitable for embedding the 200,000-sentence OpenITI index on a CPU without excessive memory usage.

An important consideration for RAG quality is *domain match*: if the retrieval corpus covers a different domain or time period than the input text, retrieved passages may not be relevant and could mislead the model. The domain mismatch between the classical OpenITI corpus and the modern newspaper PATS-A01 text is a central element of the Phase 5 analysis in Chapter 5.

2.4.5 Self-Reflective and Meta-Cognitive Prompting

Self-reflective prompting refers to techniques that incorporate information about the model's own behaviour into the prompt to guide future outputs. The Reflexion framework [24] demonstrates that language agents can improve over time by receiving verbal feedback about their past failures and incorporating this feedback into their decision-making. Similarly, self-critique prompting asks the model to evaluate its own output and revise it, akin to verbal reinforcement learning.

Phase 4D operationalises a lightweight variant of this idea. Rather than asking the model to introspect on its current output at inference time – which adds latency and the risk of circular reasoning – the model is given *pre-computed statistics* about its systematic failure patterns, derived from analysing its corrections on a separate training split. These statistics are converted to Arabic diagnostic sentences injected into the system prompt. The approach is a form of offline meta-learning: the model's

systematic biases are characterised before deployment, and this characterisation is used to nudge its inference behaviour.

This design differs from standard Reflexion in two ways: it does not require an execution environment or reward signal; and it provides population-level statistics ("you fix insertion errors 97% of the time but introduce errors on 25% of correct tokens") rather than sample-level feedback. Whether this coarser form of self-knowledge translates into improved behaviour at inference time is one of the empirical questions studied in this thesis.

2.5 Knowledge Sources Used in This Study

2.5.1 Confusion Matrix as a Knowledge Source

An OCR confusion matrix records, for each ground-truth character, the frequency with which the OCR engine substituted other characters in its place. It is a compact, data-driven summary of the engine's systematic recognition errors. Unlike generic Arabic spelling rules, a confusion matrix is engine-specific: it reflects the particular visual features that this engine's neural network has difficulty discriminating.

In this study, the confusion matrix is generated from Phase 1 analysis (Section 3.2 and Chapter 4) and is used in Phase 3 to inject the top-10 most frequent confusion pairs into the system prompt. The hypothesis is that naming these specific pairs directs the model's attention to the character positions most likely to contain errors.

2.5.2 Arabic Orthographic Rules

Arabic orthographic rules govern the correct spelling of characters that OCR commonly misidentifies. The rules used in this study address: hamza placement (al-qat' vs. al-wasl), taa marbuta vs. ha, alef maqsura vs. ya, tanwin placement, and definite article assimilation. These rules are grounded in classical Arabic grammar and are taught in Arabic primary education. They are stored in the `data/rules/`

directory as JSONL files and loaded by `RulesLoader` at runtime.

Unlike the confusion matrix (which is descriptive – "this engine makes this mistake"), orthographic rules are prescriptive – "the correct form is X". Both types of knowledge tell the model what to look for; the confusion matrix focuses its attention by describing the engine's error patterns, while the rules frame the correction task in terms of canonical Arabic spelling.

2.5.3 The QALB Corpus

The Qatar Arabic Language Bank (QALB) was developed for the QALB Shared Task on Automatic Arabic Text Correction [25, 26]. It contains thousands of human-authored Arabic sentences from online news comments, annotated with corrections for spelling, grammatical, punctuation, and other errors by trained annotators.

QALB is used in this study as a source of few-shot demonstration pairs for Phase 4B. After filtering for OCR-relevance and length (Section 3.4.3, Chapter 3), 528 pairs are available. The primary limitation of QALB as an OCR correction resource is the *transfer gap*: QALB errors are human typing errors, while OCR errors are visual confusion errors. Human typers tend to make phonological errors (writing a word as it sounds in dialectal pronunciation) or agreement errors (wrong gender or case marking), while OCR engines make visual confusion errors (confusing characters with similar shapes). The overlap between these two error distributions is partial, and the few-shot examples do not perfectly model what the model is being asked to correct.

2.5.4 The OpenITI Corpus

The Open Islamicate Texts Initiative (OpenITI) is a large-scale digitised collection of texts from the Islamicate scholarly tradition, containing classical Arabic, Persian, and other languages [27, 28]. With hundreds of millions of words of Arabic text spanning many centuries and genres, it provides a large pool of grammatically correct

Arabic sentences for dense retrieval.

OpenITI is used in Phase 5 as the retrieval corpus for RAG. Its primary limitation for this application is *domain mismatch*: OpenITI texts are classical and historical, while PATS-A01 Akhbar documents are modern newspaper text. The semantic and lexical overlap between medieval religious scholarship and contemporary news reporting is low, which affects retrieval quality. A retrieval corpus drawn from modern Arabic newspapers would be a better match for the PATS-A01 domain.

2.5.5 CAMeL Tools

CAMeL Tools [29] is an open-source Python toolkit for Arabic NLP, developed at the Computational Approaches to Modeling Language Lab at NYU Abu Dhabi. The components used in this study are:

MorphologicalAnalyzer. Given an Arabic word token, the analyser queries the MSA morphological database (`calima-msa-r13`) and returns all valid morphological analyses: each analysis specifies the lemma, root, word pattern, part-of-speech, and inflectional features (gender, number, case, tense, etc.). A word with zero analyses is morphologically invalid – it is not a recognisable MSA word form.

Application in Phase 1. CAMeL Tools is used in Phase 1 to classify OCR error tokens as either *non-words* (no valid morphological analysis) or *valid-but-wrong* (at least one valid analysis but not the correct one in context). The distinction is informative: if most OCR errors are valid-but-wrong, then dictionary lookup cannot detect them and context is required for correction.

Application in Phase 4C. The *revert strategy* described in Chapter 3 uses the morphological analyser to post-process LLM corrections. Words introduced by the LLM that fail morphological analysis while the original OCR token passed are reverted to the OCR original.

2.6 Related Work on Post-OCR Correction

2.6.1 Classical Approaches

The earliest computational approaches to post-OCR correction relied on lexicons and language models to identify and resolve implausible recognitions. Tong and Evans [2] proposed a statistical approach using a unigram word model and a character-level confusion model derived from known OCR error patterns. Their method identified likely errors by flagging tokens not found in the lexicon or whose recognition score was below a threshold, then proposed corrections by searching the lexicon for candidates within a small edit distance. This work established the noisy channel framework for OCR correction that remained influential for over a decade.

Kolak and Resnik [30] developed this framework further with an explicit noisy channel model combining character-level OCR confusion probabilities with a word-level language model. Their system formulated post-OCR correction as a decoding problem: given the OCR output, find the most probable original text under the joint model. This framing is conceptually equivalent to the machine translation noisy channel model and shares its strengths (principled probabilistic framework) and limitations (requires domain-specific language models and large training corpora of OCR errors).

The limitation common to all classical approaches is their reliance on fixed-vocabulary lexicons and short-context language models. They cannot resolve *valid-but-wrong* errors – tokens that are real words but not the intended word in context – without long-range context, and they require error-specific training data for each OCR engine being corrected.

2.6.2 Neural Sequence-to-Sequence Correction

The deep learning era brought encoder-decoder models to post-OCR correction. Dong and Smith [31] applied a multi-input attention model that combines character-

level and word-level representations of the OCR output, enabling the model to attend to both local character confusion patterns and broader word-level context. Their system significantly outperformed classical methods on historical English document collections, demonstrating that neural sequence models can implicitly learn OCR error distributions from paired training data.

Transformer-based sequence-to-sequence models have since been applied to post-OCR correction with further improvements. The ICDAR 2019 Post-OCR Correction Competition [9] attracted systems based on BERT, character-level transformers, and ensemble methods. These systems substantially improved over the OCR baseline on historical French and English text.

The fundamental challenge for these approaches in the Arabic context is the *scarcity of training data*. Neural sequence-to-sequence correction requires large quantities of paired (OCR output, ground truth) examples to learn the error model for a specific engine. Such data is relatively abundant for historical English and French (where digitised collections with corresponding scans and transcripts exist), but scarce for modern Arabic. Creating a training set for Arabic OCR correction would require either manual transcription of thousands of document scans or a data augmentation strategy that synthetically generates OCR errors. This data requirement motivates the zero-shot and prompt-based approaches investigated in this thesis, which require no paired training data.

2.6.3 LLM-Based Post-OCR Correction

The emergence of large instruction-tuned language models created a new paradigm for post-OCR correction: the model is instructed to correct errors in the OCR output without any task-specific fine-tuning. Kang et al. [32] demonstrated that ChatGPT-level models are effective at English OCR post-correction in a zero-shot setting, achieving substantial CER reductions on several historical English document benchmarks. Their results show that the implicit knowledge about spelling, grammar, and vocabulary encoded in pre-training is sufficient to correct many OCR errors without any paired training data.

This line of work directly motivates the present study. However, Kang et al. and similar work focus entirely on English. Whether the same approach generalises to Arabic – a morphologically richer script-level script with a different error distribution – is not addressed. Furthermore, no prior work has compared multiple prompt augmentation strategies under controlled conditions for Arabic OCR correction.

The phenomenon of *over-correction* – where the LLM changes tokens that the OCR rendered correctly, introducing errors rather than fixing them – is discussed qualitatively in some LLM correction work but has not been systematically studied as the dominant failure mode in the Arabic setting.

2.6.4 Arabic-Specific OCR Correction

Work specifically targeting Arabic OCR correction is limited. Several groups have applied AraBERT fine-tuned on QALB-like data to Arabic error correction, achieving competitive results on the QALB shared task. However, the QALB task targets human writing errors in online text comments rather than OCR-specific visual confusion errors, and fine-tuned models trained on QALB data are not well-calibrated for the OCR error distribution.

Rule-based approaches using Arabic spell-checkers such as Hunspell with Arabic dictionaries have also been applied to post-processing Arabic OCR output. These systems can flag non-word tokens (similar to classical approaches) but cannot resolve valid-but-wrong errors.

To the best of the author's knowledge, no published study has: (a) systematically compared multiple distinct prompt augmentation strategies for Arabic post-OCR correction in an isolated experimental design; (b) applied self-reflective prompting to the OCR correction task in any language; or (c) characterised the over-correction failure mode in the Arabic post-OCR setting with quantitative evidence.

2.6.5 Positioning of This Work

This thesis makes the following contributions relative to the existing literature:

1. **Systematic evaluation of six isolated augmentation strategies.** Prior work on LLM-based OCR correction has at most compared zero-shot with one or two augmentation conditions. This study isolates six conditions (confusion matrix, orthographic rules, few-shot examples, morphological post-processing, self-reflective prompting, and RAG), each compared against the same zero-shot baseline using the same model, dataset, and evaluation protocol.

2. **Self-reflective prompting for OCR correction.** The Phase 4D strategy – injecting pre-computed statistics about the model's own training-split error patterns into the inference prompt – is, to the author's knowledge, the first application of self-reflective prompting to OCR correction in any language.

3. **Quantitative study of over-correction in Arabic OCR.** The finding that the LLM introduces errors on 25.06% of previously correct tokens, and that this rate increases as more knowledge is injected into the prompt, is a novel empirical characterisation of a failure mode that deserves attention from practitioners deploying LLMs for text normalisation tasks.

4. **Arabic-specific evaluation.** The PATS-A01 Akhbar dataset provides a controlled typewritten Arabic testbed with known font characteristics and a clean ground truth, enabling precise attribution of CER changes to specific prompt interventions.

Chapter 3

System Design and Datasets

This chapter describes the complete experimental infrastructure underlying the study. The goal is to provide sufficient detail for full reproducibility: a reader should be able to reconstruct the pipeline, datasets, preprocessing steps, and software modules from this chapter alone. Section 3.1 introduces the three-stage pipeline architecture. Section 3.2 describes the corpora used. Section 3.3 covers preprocessing and evaluation. Section 3.4 describes the knowledge base components used across phases. Section 3.5 documents the LLM correction module. Section 3.6 presents the `PromptBuilder` and all prompt variants.

3.1 Overall Architecture

The primary constraint shaping system design is the absence of local GPU access. The research workstation can load and process text data efficiently but cannot run a 4-billion-parameter language model at useful inference speed. At the same time, cloud GPU platforms such as Kaggle and Google Colab provide free access to NVIDIA T4 GPUs that are well-suited for this model size. The architecture is designed to exploit this resource split cleanly.

3.1.1 Three-Stage Pipeline

The pipeline is divided into three stages that communicate through a single intermediary file in JSON Lines format (JSONL). Fig. 3.1 illustrates the flow.

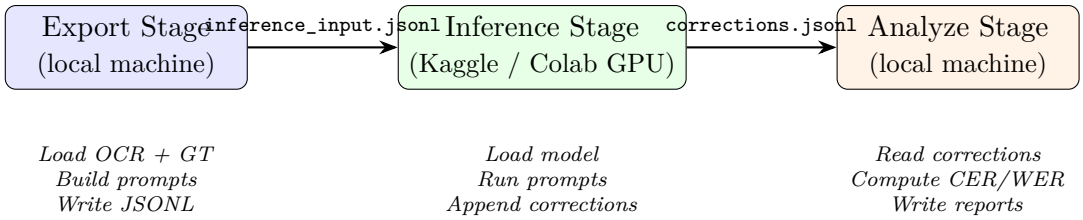

Figure 3.1: Three-stage pipeline architecture. Stages communicate via JSONL files on disk, enabling inference to run on any cloud GPU without requiring direct access to local data files.

Stage 1 — Export runs on the local machine. Each phase pipeline (e.g. `pipelines/run_phase2.py`) loads the dataset, pairs each OCR prediction with its ground-truth text, builds the phase-specific prompt for each sample using `PromptBuilder`, and writes one JSON object per sample to `inference_input.jsonl`. The file is stored under a phase- and dataset-specific results directory and is transferred to the cloud environment by cloning the repository directly.

Stage 2 — Inference runs on a Kaggle kernel or Google Colab notebook. The script `scripts/infer.py` is the unified inference entry point for all phases. It loads the model and tokenizer once at startup, then reads `inference_input.jsonl` line by line, generates a correction for each prompt, and appends the result to `corrections.jsonl`. A resume mechanism reads the output file on startup and skips any sample whose `sample_id` already appears in it, making the inference step fully restartable after an interruption without duplicating work.

Stage 3 — Analyze runs on the local machine after inference completes. The phase pipeline reads `corrections.jsonl`, computes CER and WER for each sample by comparing the corrected text against the ground truth, aggregates statistics across the dataset, and writes structured JSON reports and a human-readable Markdown summary.

3.1.2 JSONL Record Format

Each line in both JSONL files is a self-contained JSON object. Table 3.1 lists all fields and their purpose.

Table 3.1: Fields in the inference JSONL files. All fields from the export stage are preserved through inference; the `correction` and status fields are appended by the inference script.

Field	Stage added	Description
`sample_id`	Export	Unique identifier: `dataset:file_stem`
`dataset`	Export	Dataset key, e.g. `PATS-A01-Akhbar-train`
`ocr_text`	Export	Raw Qaari prediction string
`gt_text`	Export	Ground-truth text (for analysis stage only)
`prompt`	Export	Serialised chat messages list (JSON array)
`prompt_version`	Export	Version tag, e.g. `p2v2`, `p3v1`
`correction`	Inference	LLM output text
`success`	Inference	`true` if generation succeeded; else fallback used
`latency_s`	Inference	Wall-clock generation time in seconds

This design has several practical advantages. The inference machine needs no file-system access beyond the JSONL input file: all ground-truth text, phase-specific context, and metadata are embedded in each record at export time. Adding a new phase requires only implementing a new `PromptBuilder` method and a new pipeline script; the inference script and analysis logic are unchanged. Finally, the JSONL format is human-readable and can be inspected with any text editor or streamed through standard command-line tools for quick sanity checks.

3.1.3 Resume and Force-Rerun Behaviour

All three stages implement resumable execution. The export stage checks whether the target `inference_input.jsonl` already contains a record for a given dataset key before processing it, enabling partial re-runs after adding new datasets. The inference stage reads the output file on startup and populates a set of completed `sample_id` values; any sample already in the set is skipped. The analyze stage checks whether the

primary output file (`results/<phase>/<dataset>/metrics.json`) already exists before recomputing metrics. All stages accept a `-force` flag to override resume behaviour and reprocess everything from scratch. This design prevents accidental data loss while keeping re-runs cheap when only a subset of datasets changes.

3.2 Datasets

Two Arabic text datasets are included in the study. All quantitative experiments reported in this thesis use a single dataset: PATS-A01, Akhbar font, training split, 200 samples. The KHATT dataset is included in the data loading infrastructure but its evaluation is deferred to future work.

3.2.1 PATS-A01 — Printed Arabic Text Set

PATS-A01 is a publicly available Arabic typewritten text recognition dataset. It contains page images of Arabic text typeset in eight distinct fonts, together with line-level and page-level ground truth.

3.2.1.1 Fonts and Coverage

The eight fonts included in PATS-A01 are: Akhbar, Andalus, Arial, DecoType Naskh, Simplified Arabic, Tahoma, Thuluth, and Traditional Arabic. These fonts span a wide range of visual styles from the clean modern sans-serif Tahoma to the decorative classical Thuluth, providing diversity in glyph shape and rendering complexity that elicits different OCR error patterns across fonts. All experiments in this thesis use the Akhbar font, which is a common newspaper display font representative of modern typewritten Arabic.

3.2.1.2 Ground-Truth Encoding

Ground-truth files are encoded in Windows Arabic Code Page 1256 (CP-1256), a legacy single-byte encoding used widely in Arabic software on Windows systems. The DataLoader (`src/data/data_loader.py`) converts all ground-truth text to UTF-8 at load time before any downstream processing. The Traditional font uses a differently named ground-truth file (`TraditionalText.txt`) instead of the naming convention followed by the other seven fonts (`A01-{font}Text.txt`); the DataLoader handles this as a special case.

3.2.1.3 Train/Validation Split

PATS-A01 does not provide an official train/validation split. An 80/20 random split was generated using the script `scripts/generate_pats_splits.py` with a fixed random seed (42) for reproducibility. The resulting split is stored as a JSON file at `data/ocr-raw-data/PATS_A01_Dataset/pats_splits.json` and records sample identifiers assigned to either the training or validation partition. Each font yields 2,213 training samples and 553 validation samples (2,766 total per font). Each dataset configuration entry uses the field `pats_split: "train"` or `pats_split: "validation"` to select the appropriate partition at load time, ensuring that experiments that evaluate on training data are never accidentally contaminated with validation samples.

3.2.1.4 Scope of This Thesis

All quantitative experiments use the Akhbar font training split. A random sample of 200 items is drawn from this split for all phase comparisons, providing a practical compute budget (each of the 200 samples requires one model inference call) while remaining large enough for meaningful statistical analysis. Phase 4D additionally uses the validation split to analyse error patterns on held-out data, as described in Chapter 4.

3.2.2 KHATT — Khatt Arabic Handwritten Text

KHATT is an Arabic handwritten text dataset [33]. It contains text written by multiple writers in unconstrained natural handwriting, representing realistic variation in stroke style, baseline angle, character connectivity, and ink characteristics. The dataset provides separate training and validation splits. Within this project, Qaari results for KHATT are stored under `data/ocr-results/qaari-results/khatt-data/{split}/{Trainin` and the DataLoader supports them using the keys `KHATT-train` and `KHATT-validation`. Quantitative evaluation on KHATT is deferred to future work because the primary research question — whether prompt augmentation improves correction on high-quality typewritten Arabic — is best studied on a controlled, homogeneous dataset first. KHATT is expected to present a more challenging OCR baseline due to unconstrained handwriting, and it is plausible that knowledge augmentation strategies would behave differently in that setting.

3.2.3 Dataset Statistics

Table 3.2 summarises all datasets included in the system.

3.2.4 OCR Results Format

Qaari produces one plain-text `.txt` file per input image. Each file contains the recognised text for that document page. OCR results are stored under `data/ocr-results/qaari-result` with sub-directories organised by dataset type:

- PATS-A01: `qaari-results/pats-a01-data/A01-{font}/`
- KHATT: `qaari-results/khatt-data/{split}/{Training|Validation}/`

The DataLoader pairs each OCR file with its corresponding ground-truth entry by matching the file stem. The OCR engine is configurable: the configuration field `data.ocr_model` names the sub-directory within `data/ocr-results/` that holds

Table 3.2: Dataset summary. "Evaluated" is the number of samples used in experiments reported in this thesis. "—" indicates that statistics depend on writer count and session structure not yet fully catalogued.

Dataset Key	Type	Split	Total	Evaluated	Notes
PATS-A01-Akhbar-train	Typewritten	Train	2,213	200	Primary e
PATS-A01-Akhbar-val	Typewritten	Validation	553	200	Phase 4D
PATS-A01-Andalus-{train|val}	Typewritten	Both	2,766	0	Future wo
PATS-A01-Arial-{train|val}	Typewritten	Both	2,766	0	Future wo
PATS-A01-Naskh-{train|val}	Typewritten	Both	2,766	0	Future wo
PATS-A01-Simplified-{train|val}	Typewritten	Both	2,766	0	Future wo
PATS-A01-Tahoma-{train|val}	Typewritten	Both	2,766	0	Future wo
PATS-A01-Thuluth-{train|val}	Typewritten	Both	2,766	0	Future wo
PATS-A01-Traditional-{train|val}	Typewritten	Both	2,766	0	Future wo
KHATT-train	Handwritten	Train	—	0	Future wo
KHATT-validation	Handwritten	Validation	—	0	Future wo
Total PATS-A01 font-split combinations: 16 (8 fonts 2 splits)					

all results for that engine. Changing this field switches the entire pipeline to a different OCR engine's output directory without any code modification. This design anticipates future comparative studies across multiple Arabic OCR engines.

3.3 Preprocessing and Evaluation

3.3.1 Text Normalisation

Text normalisation is applied identically to both the hypothesis (OCR output or LLM correction) and the reference (ground truth) before any metric is computed. The following transformations are applied:

1. **Diacritic stripping.** Arabic short vowel marks (harakat/tashkeel) occupy Unicode code points U+064B through U+065F and related ranges (U+0610–U+061A for extended marks, U+0670 for the superscript alef). These characters are removed from both texts before computing CER and WER. The

rationale is that diacritics are stylistically optional in most modern Arabic text; their presence or absence does not change lexical identity. Penalising a corrector for changing a diacritised form to the same word without diacritics, or for failing to add diacritics that were absent in the OCR input, would conflate orthographic convention with correction quality. Both the raw metric (with diacritics) and the normalised metric (without diacritics) are computed and stored in all result files; the diacritic-stripped variant is reported as primary throughout this thesis.

2. **Tatweel removal.** The Arabic elongation character tatweel (kashida, U+0640) is a decorative stroke that extends word shapes without changing lexical identity. It is removed from both texts before metric computation.

3. **No letter normalisation.** Alef variants, hamza forms, and the alef maqsura / ya distinction are *not* normalised before evaluation. Collapsing these distinctions (e.g. mapping all alef variants to a bare alef) would mask a class of errors that the study specifically aims to measure. The confusion matrix and error taxonomy both depend on the uncollapsed character set.

Normalisation is implemented in `src/data/text_utils.py::normalise_arabic()`.

3.3.2 CER and WER Computation

3.3.2.1 Definitions

Character Error Rate (CER) measures the minimum edit distance between the hypothesis string h and the reference string r at the character level, normalised by the length of the reference:

$$\text{CER} = \frac{S_c + D_c + I_c}{N_c} \tag{3.1}$$

where S_c, D_c, and I_c are the number of character-level substitutions, deletions, and insertions in the minimum-edit alignment, and N_c is the total number of characters

in the reference. Word Error Rate (WER) applies the same formula at the word level, treating whitespace-delimited tokens as the atomic unit:

$$\text{WER} = \frac{S_w + D_w + I_w}{N_w} \tag{3.2}$$

CER is the primary metric throughout this thesis. Arabic morphology makes word-level metrics more sensitive to tokenisation choices; character-level errors more directly capture the visual confusion phenomena (dot placement, shape similarity) that are the dominant error type in Arabic OCR.

3.3.2.2 Implementation

CER is computed using the `editdistance` library, which provides an efficient implementation of the Levenshtein distance. WER is computed using the `jiwer` library. Both are computed *per sample first*, then aggregated:

$$\text{CER}_i = \frac{\text{editdistance}(h_i, r_i)}{\max(|r_i|, 1)} \tag{3.3}$$

where $|r_i|$ is the reference character count for sample i. Using per-sample CER (rather than pooling all characters across the corpus) ensures that every sample contributes equally to the aggregate mean regardless of its length. This prevents the few long documents in the dataset from dominating the statistic.

Aggregate statistics reported for each dataset-phase combination are: mean CER (μ), standard deviation (σ), median, and 95th percentile (p95). The standard deviation quantifies variability across samples; the p95 captures worst-case performance and is particularly useful for identifying the impact of catastrophic OCR failures such as the repetition artifact described in Chapter 5.

3.3.3 Error Taxonomy Construction

Phase 1 classifies each substitution error into one of seven categories based on the character pair (ground-truth character, OCR character) found in the minimum-edit

alignment between ground-truth and OCR text. Table 3.3 defines each category and its detection criterion. Categories are applied in order; a substitution that matches an earlier category is not re-counted in a later one.

Table 3.3: Error taxonomy used in Phase 1 baseline analysis. Categories are ordered from most specific (dot confusion) to most general (other substitution).

Category	**Detection criterion**
Dot confusion	Substitution between characters that share the same base glyph and differ only in the number or position of diacritical dots: {ba, ta, tha, nun, ya}, {qaf, fa}, {jim, ha, kha}, {dal, dhal}.
Hamza errors	Substitution among alef variants or between hamza forms: alef, alef with hamza above, alef with hamza below, alef madda, hamza alone, hamza on waw seat, hamza on ya seat.
Taa marbuta	Substitution between taa marbuta (feminine ending) and ha, which share the same base shape and differ only by two dots.
Alef maqsura	Substitution between alef maqsura (word-final /a/ written with a ya-shaped letter without dots) and ya (with dots).
Similar shapes	Substitution between characters sharing the same skeleton but differing in secondary visual features: {ra, zay}, {dal, dhal}, {ta marbuta, ha}, {ta with extended tail (ط), dha (ظ)}.
Merged words	Fewer words in the OCR output than in the ground truth; a word boundary is missing.
Split words	More words in the OCR output than in the ground truth; a spurious word boundary is inserted.
Insertion / Deletion / Other	All remaining edit operations: character insertions, character deletions, and substitutions not classified by any of the above categories.

3.4 Knowledge Base Components

Phases 3 through 5 (and Phase 6) each inject a different type of external knowledge into the LLM prompt. All knowledge loaders are implemented in `src/data/knowledge_base.py` and share a common interface: a `load()` method

that reads source data and a `format_for_prompt()` method that returns a plain Arabic string ready for insertion into a system prompt.

3.4.1 Confusion Matrix Loader

`ConfusionMatrixLoader` reads the `confusion_matrix.json` file produced by Phase 1. The confusion matrix records, for every ground-truth character, the frequency with which Qaari substituted each other character in its place.

The loader supports two code paths. If the Phase 1 output contains a pre-computed `top_20` sorted list, it is used directly for efficiency. Otherwise, the full confusion dictionary is flattened and sorted by substitution count. The `format_for_prompt()` method selects the top-N confusion pairs (default $N = 10$, configured via `phase3.top_n` in `configs/config.yaml`) and formats them as an Arabic-language bullet list, where each item identifies a source character and its most common Qaari substitute.

A dataset with fewer than 200 total substitutions (threshold configurable as `phase3.min_substitutions`) falls back to a pooled confusion matrix aggregated across all datasets of the same type (e.g. all PATS-A01 training datasets). This prevents noise-dominated confusion matrices from individual small datasets from distorting the injection content.

3.4.2 Rules Loader

`RulesLoader` loads Arabic orthographic rules from the directory `data/rules/`, which contains JSONL files (one rule object per line) with fields for the rule text, its category, and example words illustrating the rule. The loader selects rules across three files: orthography, morphology, and syntax. The `format_for_prompt()` method formats the selected rules as a numbered instruction list in Arabic.

Table 3.4 lists the primary orthographic rule categories injected in Phase 4A. These categories were chosen because they correspond to the error types most likely to appear in Arabic typewritten OCR output, as confirmed by the Phase 1 confusion

matrix and error taxonomy.

Table 3.4: Arabic orthographic rule categories used in Phase 4A. Each category targets a specific phenomenon that both human writers and OCR engines commonly mishandle.

Rule category	**Description and OCR relevance**
Hamza al-qat' (cutting hamza)	Hamza written visibly above or below the alef in all positions. OCR frequently drops the hamza diacritic and outputs a bare alef.
Hamza al-wasl (conjunctive hamza)	An alef at the start of certain verb forms and nouns whose hamza is elided in connected speech and not written. Distinguishing these two types governs the spelling of the definite article (*al-*), some verbal nouns, and Arabic proper nouns.
Taa marbuta vs. ha	The feminine ending taa marbuta and the letter ha share the same base shape; OCR dot-placement errors cause frequent confusion between them.
Alef maqsura vs. ya	Word-final /a/ written as alef maqsura (a ya-shaped letter without dots) is frequently confused with ya (with two dots below) by OCR.
Tanwin (nunation)	Double-diacritic marking for indefinite nouns; the accusative tanwin al-fath is written as a doubled fatha above an alef, which OCR may misread or drop.
Definite article assimilation	Sun letters assimilate the lam of the definite article (*al-*); moon letters do not. This governs whether the written lam appears explicitly in the word.

3.4.3 QALB Loader and Filtering

`QALBLoader` reads the Qatar Arabic Language Bank (QALB) corpus from `data/QALB-0.9.1-Dec03-2021-SharedTasks/`. QALB contains human-authored Arabic sentences paired with manually corrected versions, annotating grammatical, spelling, and punctuation errors. The 2014 training split is used in this study.

After loading, the following filters are applied in sequence:

1. **Length filter.** Pairs with source sentences shorter than 10 characters or longer than 300 characters are discarded. The upper bound of 300 characters was determined empirically: QALB sentences average between 161 and 594 characters, so a stricter original default of 100 characters eliminated the entire corpus.

2. **Word-change filter.** Pairs where the correction modifies more than 15 words are discarded. Heavily-edited sentences are unsuitable as demonstrations because they may signal to the model that aggressive rewriting is expected.

3. **OCR-relevance filter.** A pair is retained only if its error type overlaps with known OCR error patterns: dot-group substitutions, alef/hamza variants, taa marbuta / ha swaps, or similar-shape confusions. Pairs that correct purely grammatical or agreement errors without any spelling-level change are discarded, as they could teach the model to make rewrites that OCR would never introduce.

After all three filters, 528 OCR-relevant pairs are available. At inference time, Phase 4B selects 5 pairs using a diversity-maximising strategy (`selection: "diverse"`) that ensures the selected examples span multiple error categories rather than repeating the same phenomenon.

3.4.4 OpenITI Corpus and RAG Index

`OpenITILoader` extracts clean Arabic sentences from the Open Islamicate Texts Initiative (OpenITI) corpus at `data/OpenITI/`. OpenITI is a large-scale digitised collection of Islamicate texts spanning classical Arabic, Persian, and other languages of the Islamic scholarly tradition, containing hundreds of millions of words of Arabic text. It was chosen for RAG because of its size, which allows construction of a large and varied retrieval index.

3.4.4.1 Corpus Processing

The loader reads an inventory CSV file that records the local path and metadata of each OpenITI text file. Each text file is split into candidate sentences at punctuation boundaries (full stop, question mark, exclamation mark, and the Arabic semicolon U+061B). Sentences shorter than 30 characters or longer than 300 characters are discarded. Up to 200,000 sentences are sampled from primary-status texts (those with `status: "pri"` in OpenITI metadata), using a fixed random seed (42).

3.4.4.2 FAISS Retrieval Index

The filtered sentences are embedded using the multilingual sentence encoder `sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2`, an efficient 12-layer model covering over 50 languages including Arabic. Embeddings are computed in batches of 256 and indexed in a FAISS flat inner-product index (`FlatIP`). The index is built once locally by running `python pipelines/run_phase5.py -mode build` and saved to disk.

At inference time, each OCR input text is embedded as a query and the top-3 most similar sentences are retrieved by inner-product similarity. These sentences are formatted as a numbered Arabic list and injected into the system prompt as grounding context. An important caveat on domain match is discussed in Chapter 5: OpenITI contains classical and historical Arabic, while the PATS-A01 Akhbar dataset contains modern newspaper text. The semantic overlap between these domains is low, which affects the quality of retrieved passages.

3.4.5 CAMeL Tools Integration

CAMeL Tools is an open-source Arabic NLP toolkit developed at the Computational Approaches to Modeling Language (CAMeL) Lab, NYU Abu Dhabi [29]. This study uses its morphological analysis component, backed by the Modern Standard Arabic database `calima-msa-r13`.

3.4.5.1 MorphAnalyzer

`src/linguistic/morphology.py::MorphAnalyzer` wraps the CAMeL Tools `MorphologicalAnalyzer` with two extensions:

- **LRU cache.** The morphological analyser is computationally expensive per lookup. Repeated queries for the same token are served from a Least-Recently-Used cache of size 10,000, avoiding redundant analyser calls for frequently occurring words.
- **Graceful degradation.** If `camel_tools` is not installed or the morphology database has not been downloaded, `MorphAnalyzer.analyse()` returns `None` rather than raising an exception. Downstream code that checks for `None` can continue in a degraded mode without CAMeL-based functionality.

3.4.5.2 WordValidator and the Revert Strategy

`src/linguistic/validator.py::WordValidator` implements the Phase 4C post-processing logic. For each word token in the LLM-corrected output, it applies the following decision rule:

> If the LLM correction for a word *fails* morphological analysis (zero valid analyses) *and* the corresponding OCR word *passes* morphological analysis (at least one valid analysis), revert to the OCR word.

The strategy is deliberately conservative. It can only revert a word to the OCR original; it cannot introduce a new word that neither the OCR nor the LLM produced. Consequently, the revert strategy has no downside risk on any word it touches, but its upside is limited to cases where the LLM introduces a morphologically invalid Arabic form while the OCR had a morphologically valid one. The validator records summary statistics for each text: total Arabic tokens compared, reverted count, and revert rate.

3.4.6 LLM Insights Loader

`LLMInsightsLoader` reads JSON files from `results/phase4d/insights/`, which are produced during Phase 4D's training-split analysis step. Each file records, for each error category, two statistics derived from comparing Phase 2 LLM corrections against ground truth on the training split:

- `fix_rate`: the fraction of OCR errors of that category that the LLM correctly fixed (higher is better).
- `introduction_rate`: the fraction of ground-truth tokens of that category that the LLM *incorrectly* changed (lower is better).

The loader converts these statistics into Arabic-language diagnostic sentences using the following logic: error types with `fix_rate` > 0.6 are reported as strengths; those with `fix_rate` < 0.4 are reported as weaknesses; those with `introduction_rate` > 0.05 are flagged as over-correction risks. Statistics are aggregated at the dataset-type level (PATS-A01 vs. KHATT) to smooth per-sample noise. Up to three weaknesses and two over-correction warnings are injected into the Phase 4D system prompt (configurable thresholds in `phase4d.insights`).

3.5 The LLM Correction Module

3.5.1 Class Hierarchy

All LLM backend implementations share a common abstract interface defined in `src/core/llm_corrector.py`. Fig. 3.2 shows the class hierarchy.

`BaseLLMCorrector` is an abstract base class that defines two abstract methods: `correct()` and `model_name`. Implementations must satisfy the contract that `correct()` never raises an exception and always returns a `CorrectionResult` with a non-empty `corrected_text` field. On any failure, `corrected_text` is set to the

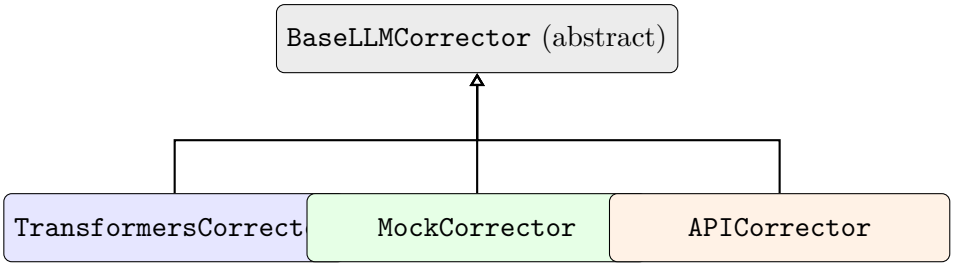

Figure 3.2: LLM corrector class hierarchy. All pipeline code depends only on the `BaseLLMCorrector` interface; the concrete backend is selected at runtime by `get_corrector(config)`.

original `ocr_text` (a safe fallback that does not inflate error rates beyond the OCR baseline).

`TransformersCorrector` is the production backend. It loads the model and tokenizer via HuggingFace Transformers once at startup and reuses them for all subsequent inference calls. It is designed to run on Kaggle or Colab GPU kernels. Optional 4-bit quantization via `bitsandbytes` is available for GPUs with less than 8 GB of VRAM, but was not used in the experiments reported here (the Kaggle T4 GPU has 16 GB).

`MockCorrector` returns the OCR text unchanged. It requires no model, no GPU, and no internet access. It is activated by setting `model.backend: "mock"` and is used for local pipeline validation: a full export–infer–analyze cycle completes instantaneously with mock, producing valid JSON outputs with CER equal to the OCR baseline (since no correction is performed).

`APICorrector` provides an OpenAI-compatible REST interface for potential future use with hosted models. It is not used in the experiments of this thesis.

3.5.2 Generation Configuration

Table 3.5 documents the exact generation parameters used in all phases of this study. All phases use identical generation settings; only the prompt content changes between phases.

Temperature 0.1 is chosen to make outputs nearly deterministic across runs while

Table 3.5: Generation parameters for `TransformersCorrector`.

Parameter	Value	Rationale
Model ID	`Qwen/Qwen3-4B-Instruct-2507`	Primary study model
`max_new_tokens`	1024	Sufficient headroom for all prompt types
`do_sample`	`True`	Stochastic sampling
`temperature`	0.1	Near-deterministic; suppresses high-entropy
`enable_thinking`	`False`	Suppresses Qwen3 chain-of-thought scratchpa
`MAX_INPUT_TOKENS`	4,096	Out-of-memory guard on unexpectedly long

avoiding the degenerate behaviour that pure greedy decoding can produce. At this temperature the output distribution is heavily peaked at the most-probable token at each step, but a small amount of stochasticity is retained to prevent the model from entering repetitive loops when correcting ambiguous text.

The `enable_thinking` flag is Qwen3-specific. When enabled, Qwen3 prepends a chain-of-thought scratchpad enclosed in `<think>...</think>` tokens before generating the final answer. For Arabic post-OCR correction, a one-pass text-to-text transformation task, the scratchpad adds latency and output tokens without improving the correction output. More critically, the scratchpad contents are sometimes included in the decoded output, breaking the response extraction step that expects only the corrected text. Setting `enable_thinking=False` in `apply_chat_template()` prevents this.

3.5.3 Response Extraction

The model is instructed to return *only* the corrected text, with no explanation or commentary. Despite this instruction, decoded outputs occasionally contain preamble or non-Arabic content. The method `_extract_corrected_text()` applies the following steps:

1. Strip leading and trailing whitespace and newlines.
2. If the result is empty, log a warning and return the original OCR text.

3. If the result contains no Arabic characters (Unicode block U+0600–U+06FF), the model has likely replied in English or refused the request; log a warning and return the original OCR text.

4. Otherwise, return the cleaned string as the correction.

Steps 2 and 3 are safety checks that prevent model refusals or empty generations from propagating as corrections. Without them, a refusal would produce a CER of approximately 1.0 for that sample, artificially degrading aggregate performance. By falling back to the OCR text on failure, the method ensures that failed samples do not inflate aggregate CER above the OCR baseline. If `max_retries` additional attempts all fail, the fallback is applied and the `success` flag in the JSONL record is set to `false`.

3.6 PromptBuilder

`src/core/prompt_builder.py::PromptBuilder` is the single point of contact for all prompt construction in the study. Each experimental phase has exactly one corresponding `build_*` method. All methods return a list of dictionaries in OpenAI chat format — a system message followed by a user message — which is directly accepted by the HuggingFace `apply_chat_template()` API.

3.6.1 Prompt Design Philosophy

All prompts in this study are written entirely in Arabic. The decision to use Arabic rather than English for system messages was motivated by two considerations. First, the correction task is inherently Arabic-specific; using the target language avoids any cross-lingual ambiguity in instruction following. Second, Qwen3, as a multilingual model, was trained on Arabic instructions and responds more reliably when the instruction language matches the text language.

All phase prompts share a *conservative instruction*: the system message tells the

model that the input text may already be correct, that it should return the text unchanged if it finds no clear errors, and that it must not change what is already correct. This instruction counteracts a known failure mode of instruction-following LLMs: they tend to interpret any correction directive as a mandate to rewrite, even when the input is already accurate. The conservative constraint is particularly important in the PATS-A01 Akhbar setting, where the OCR baseline CER on normal samples is only 1.37% and the vast majority of tokens require no change. The conservative v2 zero-shot prompt was adopted after pilot runs showed that the original aggressive v1 prompt ("correct this text") led to unacceptably high error rates on clean samples.

3.6.2 Prompt Variants

Table 3.6 provides a complete registry of all prompt variants.

Table 3.6: Prompt variant registry. Each phase uses exactly one variant. All prompts are in Arabic. Full prompt text is given in Appendix A.

Method	Tag	Injected context	Phase
`build_zero_shot()`	p2v2	None (instruction only)	2
`build_ocr_aware()`	p3v1	Top-10 Qaari character confusion pairs	3
`build_rule_augmented()`	p4av1	Arabic orthographic rules	4A
`build_few_shot()`	p4bv1	5 QALB error-correction examples	4B
`build_self_reflective()`	p4dv1	LLM error statistics from training split	4D
`build_rag()`	p5v1	Top-3 retrieved OpenITI sentences	5
`build_combined()`	p6v1	Any subset of the above contexts	6

3.6.2.1 Zero-Shot Prompt (p2v2)

The zero-shot prompt consists of an Arabic system message and the raw OCR text as the user message. The system message establishes the model as an Arabic

text proofreader and explicitly states the conservative instruction: the OCR output may be fully correct; the model should return it exactly if no clear error is found; it must not change what is already correct; it should return only the text with no explanation. There is no injected external knowledge; the model's only guidance comes from its own training and from the instruction framing.

3.6.2.2 OCR-Aware Prompt (p3v1)

The OCR-aware prompt extends the zero-shot system message with an injected Arabic bullet list of confusion pairs from `ConfusionMatrixLoader`. Each bullet item names a ground-truth character and the character Qaari commonly substitutes for it. The system message instructs the model to pay particular attention to these character pairs when correcting. If no confusion context is available, `build_ocr_aware()` falls back transparently to the zero-shot prompt.

3.6.2.3 Rule-Augmented Prompt (p4av1)

The rule-augmented prompt replaces the confusion list with a numbered list of Arabic orthographic rules from `RulesLoader`. The system message instructs the model to observe these rules during correction. The rules address the same phenomena targeted by the confusion matrix — dot-group characters, hamza, taa marbuta, alef maqsura — but frame them as prescriptive grammar rules rather than as statistical error patterns.

3.6.2.4 Few-Shot Prompt (p4bv1)

The few-shot prompt inserts five input–output demonstration pairs from the QALB corpus into the system message. Each pair is formatted as: "Input: [erroneous sentence] / Correction: [corrected sentence]" in Arabic. The system message instructs the model to correct the new text following the same approach as the examples. The examples are selected by `QALBLoader` using the diversity strategy to span multiple error types.

3.6.2.5 Self-Reflective Prompt (p4dv1)

The self-reflective prompt injects Arabic diagnostic sentences derived from the model's own training-split error statistics. The system message frames the diagnostics as performance feedback: the model is informed which error categories it handles well, which it tends to miss, and for which categories it tends to over-correct previously correct tokens. This is a lightweight, pre-computed form of meta-cognitive guidance that avoids the latency cost of asking the model to self-critique at inference time.

3.6.2.6 RAG Prompt (p5v1)

The RAG prompt injects three sentences retrieved from the OpenITI FAISS index. The retrieved sentences are presented as reference Arabic text that the model should use as grounding context. The system message states that these reference sentences are correct Arabic text similar to the input and that the model should consult them when deciding how to correct the input.

3.6.2.7 Combined Prompt (p6v1)

`build_combined()` accepts any subset of the five context types and assembles them into a single system message using a fixed section ordering numbered with Arabic ordinals ("First: confusions, Second: rules, Third: examples, Fourth: retrieved passages, Fifth: performance notes"). Only non-empty context strings are included; the closing conservative instruction is always appended. If all context strings are empty, the method falls back to the zero-shot prompt. This design makes Phase 6 combination experiments straightforward to specify and fully reproducible from the combination identifier alone.

Chapter 4

Experimental Methodology

This chapter describes the experimental methodology of each phase. For every phase, the structure follows a consistent template: hypothesis, method (step-by-step), variables, output files, and the research question answered. Section 4.1 explains the overarching design principles that govern how the phases relate to each other. Sections 4.2 through 4.10 describe each phase in full. Table 4.3 at the end of this chapter provides a consolidated reference.

4.1 Experimental Design Principles

4.1.1 The Isolation Principle

The defining design choice of this study is that Phases 3 through 5 are *isolated* experiments. Each phase changes exactly one variable relative to Phase 2: the type of knowledge injected into the system prompt. Every other factor — the model, the dataset, the sample set, the generation parameters, the evaluation protocol, the analysis code — is held fixed. This design allows any observed CER change to be attributed unambiguously to the knowledge injection and not to a confound.

Comparing isolated phases against Phase 2 (zero-shot) rather than against Phase 1 (OCR baseline) is a deliberate choice. The question of interest is not whether the LLM improves on raw OCR (Phase 2 answers that), but whether injecting a specific type of knowledge into an already-running LLM corrector improves its performance.

Comparing to Phase 1 would confound OCR quality with LLM correction quality.

4.1.2 Phase 2 as the Hub Baseline

Phase 2 (zero-shot LLM correction) serves as the comparison baseline for all subsequent phases. A phase that does not improve on Phase 2 is a negative result with respect to that knowledge type: the injected knowledge does not help, and may actively hurt, relative to the model's own language understanding. Negative results are scientifically valuable because they rule out a hypothesis and constrain theories of what makes LLM correction succeed.

4.1.3 Reproducibility Measures

All experiments use:

- The same 200-sample random draw from PATS-A01 Akhbar training split.
- The same model: `Qwen/Qwen3-4B-Instruct-2507`.
- The same generation parameters: temperature 0.1, `max_new_tokens` 1024, `enable_thinking=False`.
- The same evaluation protocol: diacritic-stripped CER/WER, per-sample then aggregated.
- Fixed random seeds where any sampling is involved (seed 42 throughout).

Each phase stores its inference input as a JSONL file with embedded prompts, ensuring that the exact prompts sent to the model are permanently recorded and can be inspected or re-run.

4.1.4 Why Not Test All Combinations First

With five distinct knowledge sources (confusion matrix, rules, few-shot, self-reflective insights, RAG), the full combination space contains $2^5 = 32$ subsets. Running all 32 on the full 200-sample dataset would require approximately 6,400 model inference calls plus the same number of analysis pipeline runs, well beyond the available compute budget. More importantly, running 32 uninformed combinations before understanding each component's individual behaviour would make the results difficult to interpret: a combination that degrades performance could not be decomposed into its contributing factors without the isolated phase results.

The hierarchical strategy (isolated first, then best combinations in Phase 6) extracts the maximum insight per inference call: isolated phases diagnose each component, and Phase 6 tests whether synergies exist between the best performers.

4.2 Phase 1 — Baseline and Error Taxonomy

4.2.1 Hypothesis

Qaari produces systematic, categorisable errors on Arabic typewritten text. The error profile is dominated by a small number of recurring character confusion types that can be quantified and used to guide subsequent phases.

4.2.2 Method

1. Load all 200 OCR predictions and ground-truth pairs for PATS-A01 Akhbar training split from the DataLoader.

2. Compute per-sample CER and WER using the normalised metric (diacritics stripped). Aggregate: mean, standard deviation, median, p95.

3. Build a character-level confusion matrix. For each aligned (ground-truth

character, OCR character) pair in the minimum-edit alignment, increment the count for that pair. Normalise by ground-truth character frequency to obtain conditional confusion probabilities.

4. Classify each substitution, insertion, and deletion into one of the seven taxonomic categories from Table 3.3 (Chapter 3) using the heuristic rules defined there.

5. Apply CAMeL Tools morphological analysis to each unique OCR token. Classify tokens as *non-words* (zero valid analyses) or *valid-but-wrong* (at least one valid analysis, but contextually incorrect).

6. Write per-dataset reports including confusion matrix JSON, error taxonomy JSON, and a human-readable Markdown summary.

4.2.3 Output Files

- `results/phase1/{dataset}/metrics.json` — CER/WER statistics
- `results/phase1/{dataset}/confusion_matrix.json` — character-level confusion counts with top-20 list
- `results/phase1/{dataset}/error_taxonomy.json` — error counts and percentages by category with examples
- `results/phase1/{dataset}/morphological_analysis.json` — word validity statistics
- `results/phase1/{dataset}/error_examples.json` — up to 5 sample OCR/GT pairs per error category
- `results/phase1/{dataset}/report.md` — human-readable summary

Research question answered: RQ1.

4.3 Phase 2 — Zero-Shot LLM Correction

4.3.1 Hypothesis

A zero-shot instruction-tuned LLM can improve Arabic OCR output without task-specific guidance, by exploiting its pre-trained Arabic language knowledge to recognise and correct implausible character sequences.

4.3.2 Method

1. For each sample in the 200-sample evaluation set, construct the conservative zero-shot prompt using `PromptBuilder.build_zero_shot(ocr_text, version="v2")`. The system message tells the model that the text may already be correct, that it should return it unchanged if no clear error is found, and that it must not change what is already correct.

2. Write all prompts to `results/phase2/inference_input.jsonl`, one JSON object per sample, containing fields: `sample_id`, `dataset`, `ocr_text`, `gt_text`, `prompt`, `prompt_version` (= `p2v2`).

3. Run inference on Kaggle (GPU) using `scripts/infer.py`. For each sample, the model generates the corrected text. Results are written to `results/phase2/corrections`

4. Compute post-correction CER and WER for each sample. Compare to Phase 1 OCR baseline. Compute error changes: for each sample, count errors fixed (present in OCR but absent from correction) and errors introduced (absent from OCR but present in correction).

5. Write comparison report to `results/phase2/{dataset}/comparison_vs_phase1.json`.

4.3.3 Prompt Design Rationale

Two prompt versions were developed:

- **v1 (aggressive)**: "You are an Arabic text corrector. Your task is to correct OCR errors in Arabic text. Return only the corrected text without explanation." (translated from Arabic)
- **v2 (conservative)**: "You are an Arabic text proofreader. The following text is OCR output and may be fully correct or contain few errors. If the text is correct, return it exactly as-is. If you find only a clear error, correct it. Do not change what is correct. Return only the text without explanation." (translated from Arabic)

Version v1 was piloted but discarded after it produced worse aggregate CER than the OCR baseline on clean samples: the aggressive framing caused the model to rewrite text it should have preserved. Version v2 is used in all experiments reported in this thesis.

4.3.4 Output Files

- `results/phase2/{dataset}/metrics.json`
- `results/phase2/{dataset}/comparison_vs_phase1.json`
- `results/phase2/{dataset}/error_changes.json`

Research question answered: RQ2. **Role in the study:** Phase 2 output serves as the comparison baseline for all subsequent phases.

4.4 Phase 3 — OCR-Aware Prompting

4.4.1 Hypothesis

Telling the LLM which specific character confusions Qaari makes will direct its attention to the positions most likely to contain errors and improve correction quality for those error types.

4.4.2 Method

1. Load the Phase 1 confusion matrix from `results/phase1/PATS-A01-Akhbar-train/confusi` using `ConfusionMatrixLoader`.
2. Extract the top-10 confusion pairs by substitution count. For the PATS-A01 Akhbar training split, the dominant confusions are detailed in Chapter 5.
3. Format the confusion pairs as an Arabic-language bullet list using `ConfusionMatrixLoader.f` Each bullet item reads (translated): "This system confuses [correct character] with [OCR character]."
4. Build the OCR-aware prompt using `PromptBuilder.build_ocr_aware(ocr_text, confusion_context)`. The system message extends the zero-shot message with the confusion list and instructs the model to pay particular attention to these pairs.
5. Export, infer, and analyze following the same three-stage pipeline as Phase 2.

4.4.3 Variables

- **Changed from Phase 2:** Confusion list injected into system prompt.
- **Fixed:** Model, dataset, generation parameters, evaluation protocol.
- **Number of confusion pairs:** 10 (configured as `phase3.top_n`).

4.4.4 Output Files

- `results/phase3/{dataset}/metrics.json`
- `results/phase3/{dataset}/comparison_vs_phase2.json`
- `results/phase3/{dataset}/confusion_impact.json` — per-confusion-pair analysis of whether the injection helped or hurt for samples containing that confusion

Research question answered: RQ3.

4.5 Phase 4A — Rule-Augmented Prompting

4.5.1 Hypothesis

Explicit Arabic orthographic rules provide grounded, interpretable guidance that improves correction for rule-governed phenomena such as hamza placement, taa marbuta, and alef maqsura.

4.5.2 Method

1. Load rules from `data/rules/` using `RulesLoader`. Rules are selected from the orthography, morphology, and syntax JSONL files without a category restriction (`categories: null` in config).
2. Format rules as a numbered Arabic instruction list using `RulesLoader.format_for_prompt()`. The format is: "1. [rule text in Arabic]" repeated for each rule.
3. Build the rule-augmented prompt using `PromptBuilder.build_rule_augmented(ocr_text, rules_context)`. The system message instructs the model to observe these rules during correction.
4. Export, infer, and analyze as in Phase 2.

4.5.3 Variables

- **Changed from Phase 2:** Arabic orthographic rules injected into system prompt.
- **Fixed:** Model, dataset, generation parameters, evaluation protocol.
- **Rule categories:** All available (orthography, morphology, syntax).

4.5.4 Output Files

- `results/phase4a/{dataset}/metrics.json`
- `results/phase4a/{dataset}/comparison_vs_phase2.json`
- `results/phase4a/{dataset}/error_changes.json`

Research question answered: RQ4.

4.6 Phase 4B — Few-Shot Examples

4.6.1 Hypothesis

Providing the model with demonstration pairs of erroneous input and correct output from the QALB corpus will give it a concrete template for what Arabic error correction looks like, reducing aggressive rewrites and improving targeted fixes.

4.6.2 Method

1. Load QALB-2014 training split using `QALBLoader` from `data/QALB-0.9.1-Dec03-2021-Shar`
2. Apply filters: length 10–300 characters, maximum 15 words changed, OCR-relevance (dot substitutions, alef/hamza variants, similar-shape confusions). After filtering: 528 OCR-relevant pairs available.
3. Select 5 pairs using the diversity strategy (`selection: "diverse"`, seed 42), ensuring multiple error categories are represented among the 5 examples.
4. Format examples as an Arabic inline list: "Input: [erroneous text] / Correction: [corrected text]" for each pair.
5. Build the few-shot prompt using `PromptBuilder.build_few_shot(ocr_text, examples_context)`.

6. Export, infer, and analyze as in Phase 2.

4.6.3 Transfer Gap Discussion

The QALB corpus contains errors introduced by human writers: phonological substitutions (writing a word as it sounds in a particular dialect), agreement errors (wrong gender or case marking), and punctuation errors. OCR errors are fundamentally different in kind: they are visual confusion errors caused by similar glyph shapes. The few-shot examples may teach the model to be attentive to Arabic spelling in general, but the specific error patterns they demonstrate do not closely match the error patterns produced by Qaari. This *transfer gap* is the primary structural limitation of Phase 4B and is examined quantitatively in Chapter 5.

4.6.4 Output Files

- `results/phase4b/{dataset}/metrics.json`
- `results/phase4b/{dataset}/comparison_vs_phase2.json`
- `results/phase4b/{dataset}/error_changes.json`

Research question answered: RQ5.

4.7 Phase 4C — CAMeL Morphological Post-Processing

4.7.1 Hypothesis

Morphological post-processing can detect LLM hallucinations — cases where the LLM introduces morphologically invalid Arabic words — and revert those words

to the (valid) OCR original, reducing the error introduction rate without affecting correctly corrected words.

4.7.2 Method

Phase 4C does not generate new prompts or run a new inference call. It applies the revert strategy as a post-processing step to the Phase 2 corrections.

1. Load Phase 2 corrections from `results/phase2/{dataset}/corrections.jsonl`.
2. Initialise `MorphAnalyzer` with the MSA database `calima-msa-r13`. **Note:** CAMeL Tools requires a separate installation step: `pip install camel-tools` followed by `camel_data -i morphology-db-msa-r13`.
3. For each sample, apply `WordValidator.validate_correction(ocr_text, llm_text)`:
   (1) Tokenise both the OCR text and the LLM correction into Arabic word tokens.
   (2) For each word pair (ocr_word, llm_word) that differs:
      - Run `MorphAnalyzer.analyse(llm_word)`; if result has zero analyses: LLM word is invalid.
      - Run `MorphAnalyzer.analyse(ocr_word)`; if result has at least one analysis: OCR word is valid.
      - If both conditions hold: revert to `ocr_word`.
      - Otherwise: keep the LLM correction.
   (3) Record: words reverted, words kept, revert rate.
4. Compute post-validation CER/WER on the final text. Compare to both Phase 2 (LLM only) and Phase 1 (OCR baseline).
5. Write validation statistics to `results/phase4c/{dataset}/validation_stats.json`.

4.7.3 Revert Strategy Rationale

The revert strategy is deliberately conservative. It can only revert a word to the OCR original; it cannot introduce new words. The maximum harm it can do is revert a correctly-improved LLM word back to a wrong OCR word — but only if the LLM word happened to be morphologically invalid despite being the correct intended word (which is rare for genuine corrections). The maximum benefit is preventing a hallucinated, morphologically invalid LLM word from degrading the text when the OCR had a valid form at that position.

This one-sided risk profile means Phase 4C can be deployed as a defensive layer on top of any LLM corrector with no risk of making performance worse than the raw OCR baseline.

4.7.4 Output Files

- `results/phase4c/{dataset}/metrics.json`
- `results/phase4c/{dataset}/comparison_vs_phase2.json`
- `results/phase4c/{dataset}/validation_stats.json` — per-dataset revert counts, rates, and list of reverted words

Research question answered: RQ6.

4.8 Phase 4D — Self-Reflective Prompting

4.8.1 Hypothesis

Feeding the model statistical summaries of its own systematic errors — derived from its corrections on the training split where ground truth is available — will cause

it to apply targeted corrections more carefully and reduce its error introduction rate on the validation split.

4.8.2 Three-Stage Method

Phase 4D is itself a three-stage process that must be executed in order.

4.8.2.1 Stage 1: Training-Split Analysis (analyze-train mode)

1. Load Phase 2 corrections on the training split (`PATS-A01-Akhbar-train`) from `results/phase2/PATS-A01-Akhbar-train/corrections.jsonl`.
2. For each sample, compare the Phase 2 LLM corrections to ground truth using `LLMErrorAnalyzer.analyse_sample()`. For every character alignment between LLM output and ground truth:
   - If the OCR had an error here and the LLM fixed it: increment `fixed` for that error type.
   - If the OCR was correct here and the LLM changed it incorrectly: increment `introduced` for that error type.
3. Aggregate across all training samples to obtain per-error-type statistics:
   - `fix_rate` = `fixed` / `baseline` errors of that type (fraction of existing errors the LLM corrected).
   - `introduction_rate` = `introduced` / ground-truth tokens of that type (fraction of correct tokens incorrectly changed).
4. Write aggregated statistics to `results/phase4d/insights/PATS-A01_insights.json`.

4.8.2.2 Insights Derived from Training Data

Table 4.1 reports the statistics extracted from the 200-sample PATS-A01 Akhbar training split.

Table 4.1: Phase 4D training-split analysis results for PATS-A01 Akhbar. The LLM is Qwen3-4B-Instruct-2507 under the zero-shot v2 prompt (Phase 2 setting). Fix rate = fraction of baseline errors of that type that the LLM corrected. Introduction rate = fraction of correct ground-truth tokens of that type that the LLM incorrectly changed. $n < 10$ entries are marked with † (too few samples for reliable estimates).

Error type	**Baseline**	**Fixed**	**Introduced**	**Fix rate**	**Intro. rate**
Insertion	1,964	1,912	356	97.35%	18.13%
Other subst.	49	8	80	16.33%	163.27%
Deletion	32	1	68	3.12%	212.50%
Dot confusion	14	1	2	7.14%	14.29%
Alef maqsura	3†	1	3	33.33%	100.00%
Similar shape	1†	1	8	100.00%	800.00%
Hamza	0	—	0	—	—
Taa marbuta	0	—	0	—	—
Overall	**2,063**	**1,924**	**517**	**93.26%**	**25.06%**

† Fewer than 10 baseline samples; estimate unreliable.

The table reveals two important patterns. First, the insertion fix rate of 97.35% shows that the LLM is highly effective at collapsing the Qaari repetition artifact: almost all spurious character insertions (which constitute 95.2% of all baseline errors) are removed. Second, the overall introduction rate of 25.06% — one in four previously correct tokens is incorrectly changed by the LLM — represents a systematic over-correction bias that affects all error categories. The deletion and other-substitution introduction rates exceed 100% because the LLM introduces more deletions and substitutions of those types than existed in the original OCR output.

4.8.2.3 Stage 2: Export (export mode)

1. Load the aggregated insights from `results/phase4d/insights/PATS-A01_insights.json` using `LLMInsightsLoader`.

2. Apply the insight filtering logic:

   - Error types with `fix_rate` > 0.6: reported as strengths in the diagnostic.
   - Error types with `fix_rate` < 0.4 and `baseline` ≥ 10: reported as weak-

nesses.

- Error types with `introduction_rate` > 0.05 and `baseline` ≥ 10: reported as over-correction risks.

3. Convert the filtered statistics to Arabic-language diagnostic sentences. The resulting insights text injected into the Phase 4D validation prompt is provided in full in Appendix B. Its content identifies insertions as a strength, deletions and other substitutions as weaknesses, and the high overall introduction rate as an over-correction warning.

4. Build the self-reflective prompt for each validation-split sample using `PromptBuilder.build_s` `insights_context)`.

5. Write to `results/phase4d/inference_input.jsonl`.

4.8.2.4 Stage 3: Analyze (analyze mode)

1. Run inference on Kaggle GPU using `scripts/infer.py`.

2. Load `results/phase4d/corrections.jsonl`.

3. Compute CER/WER; compare to Phase 2.

4. Write reports to `results/phase4d/{dataset}/`.

4.8.3 Output Files

- `results/phase4d/insights/PATS-A01_insights.json` — training-split statistics

- `results/phase4d/{dataset}/metrics.json`

- `results/phase4d/{dataset}/comparison_vs_phase2.json`

- `results/phase4d/{dataset}/error_changes.json`

Research question answered: RQ7.

4.9 Phase 5 — Retrieval-Augmented Generation

4.9.1 Hypothesis

Providing the model with semantically similar passages from a large Arabic corpus (OpenITI) supplies lexical and syntactic grounding that helps it determine the correct form of ambiguous OCR sequences.

4.9.2 Method

Phase 5 is itself a three-mode pipeline (build, export, analyze).

4.9.2.1 Build Mode

1. Load OpenITI corpus inventory from `data/OpenITI/` using `OpenITILoader`.
2. Extract up to 200,000 sentences from primary-status texts, applying filters: minimum 30 characters, maximum 300 characters. Sentence splitting is performed at punctuation boundaries (full stop, question mark, exclamation, Arabic semicolon U+061B).
3. Embed all sentences using `paraphrase-multilingual-MiniLM-L12-v2` in batches of 256 on CPU.
4. Build a FAISS `FlatIP` index from the embeddings. Save index and sentence list to disk.

4.9.2.2 Export Mode

1. Load the FAISS index using `RAGRetriever`.
2. For each of the 200 evaluation samples, embed the OCR text as a query.
3. Retrieve the top-3 most similar sentences by inner-product similarity.

4. Format the retrieved sentences as a numbered Arabic list.

5. Build the RAG prompt using `PromptBuilder.build_rag(ocr_text, retrieval_context)`

6. Write to `results/phase5/inference_input.jsonl`, embedding the retrieved text in each JSONL record so no index access is needed at inference time.

4.9.2.3 Analyze Mode

1. Run inference on Kaggle GPU using `scripts/infer.py`.

2. Load `results/phase5/corrections.jsonl`.

3. Compute CER/WER; compare to Phase 2.

4. Compute retrieval quality metrics from the embedded retrieval scores: average top-1 inner-product score, average top-k score, distribution of scores across quality bands. Write to `results/phase5/{dataset}/retrieval_analysis.json`.

4.9.3 Retrieval Quality

The inner-product similarity scores between the OCR queries and the retrieved OpenITI sentences are high in absolute terms: the average top-1 score across 200 samples is 0.805 and the average top-3 score is 0.793. However, these scores reflect embedding-space proximity as computed by `MiniLM-L12-v2`, not semantic relevance to the correction task. High scores indicate that the retrieved sentences are "similar" in embedding space to the query, but this similarity may be surface-level (both texts are Arabic and share common function words) rather than meaningful semantic overlap between the retrieved classical text and the modern newspaper query. Chapter 5 connects the retrieval score distribution to the observed CER outcome.

4.9.4 Output Files

- `results/phase5/{dataset}/metrics.json`
- `results/phase5/{dataset}/comparison_vs_phase2.json`
- `results/phase5/{dataset}/retrieval_analysis.json` — per-sample retrieval scores and aggregate statistics

Research question answered: RQ8.

4.10 Phase 6 — Combinations and Ablation

4.10.1 Purpose

Phases 3–5 establish which knowledge types help and which hurt in isolation. Phase 6 asks two follow-up questions: (a) can two knowledge types combine synergistically to produce a result better than either individually? (b) in the best combined system, what is each component's quantitative contribution?

4.10.2 Combinations Tested

The 12 combinations are selected based on the isolated phase results. The guiding principle is that combinations involving components that performed similarly to Phase 2 in isolation are more likely to combine beneficially than combinations involving highly disruptive components. Table 4.2 lists all 12 combinations.

4.10.3 Ablation Design

The ablation study starts from the best-performing combination identified in Phase 6 and removes one component at a time. The CER delta when a component is removed measures that component's individual contribution to the combined system:

Table 4.2: Phase 6 combination inventory. Each combination is identified by a unique ID. Components: C = confusion matrix, R = rules, E = examples (few-shot), S = self-reflective insights, G = RAG, M = CAMeL post-processing.

Combo ID	Components	Rationale
`conf_rules`	C + R	Confusion + rules: both target character-level error
`conf_few`	C + E	Confusion + examples: data-driven + example-driv
`rules_few`	R + E	Rules + examples: best two augmented phases
`few_self`	E + S	Examples + self-reflective: both are evidence-based
`conf_rules_few`	C + R + E	Top-3 triple
`rules_few_self`	R + E + S	Alternative triple
`full_no_rag`	C + R + E + S	Full system minus RAG (best guess given phase res
`full_system`	C + R + E + S + G	All five knowledge types
`conf_few_camel`	C + E + M	Pair + CAMeL post-processing
`rules_few_camel`	R + E + M	Best pair + CAMeL
`few_self_d4d`	E + S (Phase 4D)	Self-reflective as add-on
`full_camel`	C + R + E + S + M	Full system with CAMeL, no RAG

$$\Delta_{\text{CER}}^{(k)} = \text{CER}_{\text{full}\setminus k} - \text{CER}_{\text{full}} \tag{4.1}$$

A positive $\Delta_{\text{CER}}^{(k)}$ means removing component k increases CER — the component was helping. A negative $\Delta_{\text{CER}}^{(k)}$ means removing it decreases CER — the component was hurting. This design answers RQ9: which combination is optimal and what does each element contribute?

4.10.4 Statistical Testing

For Phase 6 results, paired t-tests are used to assess whether observed CER differences between the best combination and Phase 2 are statistically significant. The Bonferroni correction is applied to control the family-wise error rate across 12 combination comparisons at the $\alpha = 0.05$ level. Bootstrap confidence intervals (1,000 iterations) are also computed for the best-performing combination. These tests are implemented in `src/analysis/stats_tester.py::StatsTester`.

4.10.5 Output Files

- `results/phase6/{combo_id}/{dataset}/metrics.json`
- `results/phase6/{combo_id}/{dataset}/comparison_vs_phase2.json`
- `results/phase6/summary.json` — all combinations ranked by CER
- `results/phase6/ablation.json` — ablation delta table
- `results/phase6/paper_tables.md` — formatted tables for publication

Research question answered: RQ9.

4.11 Phase Summary

Table 4.3 provides a consolidated reference for all phases.

Table 4.3: Summary of all experimental phases. "Comparison" is the phase each experiment is evaluated against.

Phase	Knowledge injected	Comparison	RQ
Phase 1	None (OCR analysis)	N/A	RQ1
Phase 2	None (zero-shot)	Phase 1	RQ2
Phase 3	Confusion matrix (top-10 pairs)	Phase 2	RQ3
Phase 4A	Arabic orthographic rules	Phase 2	RQ4
Phase 4B	5 QALB few-shot examples	Phase 2	RQ5
Phase 4C	CAMeL morphological post-processing	Phase 2	RQ6
Phase 4D	LLM training-split error statistics	Phase 2	RQ7
Phase 5	Top-3 OpenITI RAG passages	Phase 2	RQ8
Phase 6	Combinations + ablation	Phase 2 + best combo	RQ9

Chapter 5

Results and Analysis

This chapter reports the numerical results for all executed experimental phases (Phases 1 through 5), answers each research question stated in Chapter 1.6, and provides analysis of both expected and unexpected findings. Phase 6 results are deferred to future work as the combination experiments had not completed execution at the time of writing. Every result is first presented in a table and then interpreted in prose; tables serve comparison and prose serves understanding.

5.1 Phase 1 Results — OCR Error Characterisation

5.1.1 Aggregate Error Rates

Table 5.1 reports the CER and WER computed over the 200-sample PATS-A01 Akhbar training evaluation. Two views are given: the *all-samples* view includes every sample in the 200-sample set; the *normal-samples* view excludes the single runaway sample.

Table 5.1: Phase 1 OCR baseline metrics on PATS-A01 Akhbar (200 samples).

Sample Set	n	**CER**	**WER**	**CER Std**	**CER Median**
All samples	200	17.01%	20.14%	2.21	0.00%
Normal samples	199	1.37%	5.04%	0.02	0.00%

The striking discrepancy between the two rows — 17.01% versus 1.37% CER — is caused by a single sample that exhibits Qaari's *repetition artefact*: the OCR engine enters an infinite repetition loop on a particular text segment, generating thousands of copies of a short phrase. That one sample (0.5% of the set) contributes 1,964 of the 2,063 total character errors. Including it maintains *ecological validity*: the repetition bug is real-world behaviour of the engine and appears in any large-scale deployment. All subsequent phases are therefore evaluated on the full 200-sample set so that the ability of the LLM to collapse these repetitions is captured.

The all-samples CER standard deviation of 2.21 and a median of 0.00% reveal a heavily skewed distribution. Over half of the 200 samples have zero OCR errors; the mean is pulled upward by the single runaway and by a small number of difficult samples. On normal samples the picture is very different: the standard deviation drops to 0.02 and the CER of 1.37% reflects the genuine character-level accuracy of Qaari on clean typewritten Akhbar text.

5.1.2 Error Type Distribution

Table 5.2 breaks down the 2,063 character errors by the eight-category taxonomy defined in Section 3.3.3.

Table 5.2: Phase 1 error taxonomy on PATS-A01 Akhbar (all 200 samples, 2,063 total character errors).

Error Type	**Count**	**Percentage**
Insertion	1,964	95.20%
Other substitution	49	2.38%
Deletion	32	1.55%
Dot confusion	14	0.68%
Alef maqsura	3	0.15%
Similar shape	1	0.05%
Taa marbuta	0	0.00%
Hamza	0	0.00%

Insertions dominate at 95.2% because they are overwhelmingly produced by the single runaway sample. When that sample is set aside and only the 199 normal samples are considered, the error profile shifts substantially. Substitution errors

(dot confusion, alef maqsura, other substitution) become the dominant category, representing the genuine confusion patterns of the OCR engine on typewritten Arabic. Deletions (1.55% overall) represent characters present in the ground truth but absent from the OCR output; these tend to cluster in closely spaced character groups where segmentation fails.

The absence of hamza and taa marbuta errors is notable. These phenomena account for a significant fraction of human spelling errors in Arabic but are well-handled by Qaari's recognition model on the Akhbar font, which has unambiguous glyph renderings for these characters.

5.1.3 Confusion Matrix Analysis

The confusion matrix records every (ground-truth character, OCR character) substitution pair observed across the 200 samples. Table 5.3 presents all nine unique confusion pairs identified, ordered by frequency.

Table 5.3: Character-level confusion pairs from Phase 1 (all 9 unique pairs; out of 61 total substitutions).

GT Char	OCR Char	Count	Probability	Category
ط	ظ	8	0.89	Dot confusion
ي	ب	3	0.50	Dot confusion
ب	ي	3	0.75	Dot confusion
ى	ي	2	0.67	Alef maqsura
ا	ـ	2	0.67	Other substitution
ي	حَ	2	0.33	Similar shape
ف	ق	2	0.50	Dot confusion
ئ	ث	2	0.40	Dot confusion
م	ل	2	0.40	Similar shape

The dominant confusion, ط → ظ, accounts for 8 of the 61 total substitutions (probability 0.89). Both characters are emphatic consonants sharing a similar skeletal shape; the only distinguishing feature is a small diacritic-like mark that can be rendered ambiguously at low resolution. The bidirectional ba/ya confusion (ب ↔ ي) reflects the identical base shapes of these two letters and their reliance on dot placement — below the baseline for ب and above the baseline for ي; a slight ink

irregularity shifts the engine's decision. The alef maqsura confusion (ى → ي) is structurally identical to the classical NLP challenge described in Section 2.1.2: both characters are written without dots in many fonts and differ only in their Unicode code point.

The fen/qaf confusion (ف → ق) illustrates the dot-count ambiguity: ف has one dot above, ق has two dots above; the difference is a single ink dot. Collectively, dot-related confusions (ظ/ط, ب/ي, ي/ى, ق/ف, ث/ئ) account for 18 of the 61 substitutions (29.5%), confirming that dot ambiguity is the primary substitution driver in the Akhbar font.

5.1.4 Morphological Validity Analysis

CAMeL Tools MSA morphological analysis was applied to every token in the 200-sample set. The results are unambiguous: *100% of OCR errors are valid-but-wrong.* The confusion matrix shows zero non-word errors — every substituted or deleted character produces a token that is itself a valid Arabic word, merely the wrong one. The OCR word validity rate is 100%: all 2,801 OCR tokens are morphologically valid Arabic forms.

This finding has a profound implication for Phase 4C (morphological post-processing). A revert strategy that identifies LLM-introduced morphological invalids and reverts them to the (valid) OCR original cannot operate in this regime: the OCR errors are already valid words, so simple dictionary lookup or morphological validity checks provide no discriminative signal. Correcting valid-but-wrong substitutions requires context, not just lexical validity.

Answer to RQ1: Qaari produces systematic errors on the Akhbar typewritten corpus. The dominant failure mode is a catastrophic repetition artefact (0.5% of samples, responsible for 95.2% of all character errors) which inflates aggregate CER to 17.01%. On the remaining 199 normal samples, the CER is 1.37%, indicating high-quality recognition on clean typewritten text. The dominant substitution error type is dot confusion (29.5% of substitutions), driven by near-identical letter shapes.

All OCR errors produce morphologically valid Arabic forms, precluding detection by dictionary lookup.

5.2 Phase 2 Results — Zero-Shot LLM Baseline

5.2.1 Aggregate Performance

Table 5.4 reports the Phase 2 results. After zero-shot LLM correction with Qwen3-4B-Instruct-2507, the aggregate CER falls from 17.01% to 5.77% and WER from 20.14% to 10.60%.

Table 5.4: Phase 2 zero-shot LLM correction metrics on PATS-A01 Akhbar (200 samples). Phase 1 all-samples figures are shown for reference.

Phase	**CER**	**WER**	**CER Std**	**CER Median**	**CER P95**
1 (all samples)	17.01%	20.14%	2.21	0.00%	6.45%
2 (zero-shot LLM)	5.77%	10.60%	0.12	3.28%	19.57%

5.2.2 Comparison to Phase 1

The transition from Phase 1 to Phase 2 must be interpreted from two different reference points.

5.2.2.0.1 Relative to all-samples OCR (CER 17.01%). The LLM achieves a 66% relative CER reduction, falling from 17.01% to 5.77%. This improvement is substantially driven by the repetition artefact sample. The LLM collapses the thousands of repeated characters into a coherent corrected text, drastically reducing the edit distance to the ground truth. This is the expected strength of a language model: it recognises the output as degenerate and reconstructs the probable source sentence. The Phase 4D training-split analysis (Section 5.3.5) later quantifies this precisely: the LLM fixes 97.35% of insertion errors, corroborating its near-perfect handling of the repetition artefact.

5.2.2.0.2 Relative to normal-samples OCR (CER 1.37%). The LLM *increases* CER from 1.37% to 5.77% on the 199 normal samples — a 321% relative increase. This is the *over-correction problem* stated numerically. On text that Qaari rendered correctly, the LLM introduces errors at a rate that exceeds the errors it fixes. The model changes tokens that are already correct, and each unsolicited change has a high probability of being wrong (since the token was already right). This finding recurs throughout Chapters 5 and 6 as the central obstacle to knowledge-augmented correction on high-quality typewritten input.

5.2.3 Per-Sample Analysis

The Phase 2 CER standard deviation (0.12) is 21 times the mean (0.058), indicating a highly skewed distribution. The CER median is 3.28%, lower than the mean, confirming that the majority of samples cluster at low CER while a tail of high-CER samples pulls the mean upward. The 95th-percentile CER of 19.57% shows that the worst 10 samples (5%) incur substantial error rates, likely including the repetition sample and cases where the LLM rewrote extensive portions of correct text.

The shift in the median from 0.00% (Phase 1 all-samples) to 3.28% (Phase 2) reveals the over-correction problem at the sample level: the modal sample was error-free in the OCR output but is no longer error-free after LLM correction. Phase 2 produces a worse median outcome than the original OCR on normal samples, even though it produces a better aggregate CER by eliminating the catastrophic repetition artefact.

Answer to RQ2: A zero-shot instruction-tuned LLM can substantially reduce aggregate CER (66% relative reduction) by collapsing OCR repetition artefacts. However, on high-quality typewritten input where the baseline CER is 1.37%, the LLM introduces more errors than it fixes, raising the CER to 5.77% on normal samples. This over-correction effect defines the ceiling for all subsequent augmentation phases: no augmentation strategy can succeed if the LLM is already introducing more errors than exist in the input.

5.3 Phases 3–5 Results — Knowledge Augmentation Evaluation

Table 5.5 presents the complete comparison across all executed phases. This is the principal result table of the thesis. All phases run on the same 200-sample PATS-A01 Akhbar training split. The relative CER change column reports the percentage change versus the Phase 2 baseline; positive values indicate degradation.

Table 5.5: Main results: CER and WER for Phases 1–5 on PATS-A01 Akhbar (200 samples). Relative change vs. Phase 2 zero-shot baseline.

Phase	**Method**	**CER**	**WER**	**Std**	**Median**	**ΔCER vs P2**
1 (all)	OCR Baseline	17.01%	20.14%	2.21	0.00%	—
1 (norm)	OCR Baseline	1.37%	5.04%	0.02	0.00%	—
2	Zero-Shot LLM	5.77%	10.60%	0.12	3.28%	baseline
3	Confusion Matrix	19.62%	24.35%	2.19	1.67%	+240%
4A	Rule-Augmented	11.03%	17.70%	0.19	6.40%	+91%
4B	Few-Shot (QALB)	10.09%	15.64%	0.18	6.45%	+75%
4C	CAMeL Post-Proc.	5.77%	10.60%	0.12	3.28%	0%
4D	Self-Reflective	10.48%	16.12%	0.24	6.45%	+82%
5	RAG (OpenITI)	22.10%	26.93%	1.99	5.04%	+283%

The central finding is clear: *no knowledge augmentation strategy improves upon the zero-shot baseline on this dataset.* Every active augmentation (Phases 3, 4A, 4B, 4D, and 5) increases CER relative to Phase 2. The magnitudes range from +75% (Phase 4B, the best augmented result) to +283% (Phase 5, the worst). Phase 4C has no effect because its revert condition is never triggered. Each phase's result is now analysed in detail.

5.3.1 Phase 3 — Confusion Matrix Injection

Phase 3 achieves a CER of 19.62% and WER of 24.35%, representing a 240% relative CER increase over Phase 2. This is the second-worst outcome across all augmentation conditions.

The confusion list injected into the prompt contains the top-10 character confusion

pairs derived from the Phase 1 training data (Section 3.4.1). On a corpus where dot confusions account for only 0.68% of all errors and 29.5% of substitutions (Table 5.2), this injection creates a signal-to-noise problem. The model receives a list of characters it should scrutinise — ظ/ط, ب/ي, and so on — and interprets this as evidence that the text is broadly unreliable in all dot-group positions. In response, it substitutes characters at dot-ambiguous positions that it would otherwise leave untouched.

The CER standard deviation of 2.19 (compared to 0.12 for Phase 2) confirms that the confusion injection does not uniformly degrade all samples. Most samples remain similar to Phase 2, but a small fraction of samples are catastrophically rewritten. This bimodal effect suggests the model latches onto the confusion information selectively: when it encounters an ambiguous passage where multiple characters match the warned pairs, it aggressively substitutes; when the passage is unambiguous, the confusion list has no effect. The Phase 1 CER standard deviation of 2.21 is almost identical to Phase 3's 2.19 — the confusion injection has restored the same runaway-like variance that the repetition artefact caused in Phase 1, but through LLM over-correction rather than OCR failure.

Answer to RQ3: Injecting OCR-specific confusion information does not improve correction and produces the second-largest degradation (+240% CER relative to Phase 2). The confusion list amplifies over-correction on dot-ambiguous characters rather than directing the model's attention productively.

5.3.2 Phase 4A — Rule-Augmented Prompting

Phase 4A achieves a CER of 11.03% and WER of 17.70%, representing a 91% relative CER increase over Phase 2. This is the third-worst outcome.

The Arabic orthographic rules injected into the prompt (Section 3.4.2) cover phenomena that occur broadly throughout Arabic text: hamza al-qat', hamza al-wasl, taa marbuta, alef maqsura, and related forms. Unlike the confusion matrix (which targets a narrow set of substitution pairs), the rules are *prescriptive for all text.* The

model applies them even to tokens that were already correct, over-normalising forms that are contextually appropriate. For example, a rule about the appropriate form of hamza may cause the model to rewrite a correctly rendered hamza form because the rule's surface specification matches a token that is correct in context.

The CER standard deviation of 0.19 (higher than Phase 2's 0.12 but far below Phase 3's 2.19) indicates more uniform degradation: the rules affect all samples rather than selectively catastrophising a few. This is consistent with the nature of the augmentation: prescriptive rules apply broadly, whereas confusion pairs apply selectively.

The median CER rises from 3.28% (Phase 2) to 6.40% (Phase 4A), confirming that the modal sample is worse, not just the tail. The typical Akhbar sentence contains characters subject to the orthographic rules, so most samples incur additional errors.

Answer to RQ4: Explicit Arabic orthographic rules do not improve correction (+91% CER relative to Phase 2). Their broad applicability causes the model to over-normalise correct tokens at a higher rate than it corrects genuine OCR errors.

5.3.3 Phase 4B — Few-Shot Examples

Phase 4B achieves a CER of 10.09% and WER of 15.64%, representing a 75% relative CER increase over Phase 2. This is the *best result among all augmentation conditions*, though it still degrades relative to the zero-shot baseline.

The few-shot examples from the QALB corpus constrain the model's behaviour more precisely than abstract rules or character lists, because they demonstrate specific input–output transformations rather than general principles. The model can infer from the examples what magnitude of change is expected (minor lexical corrections, not full rewrites), which partially moderates the over-correction tendency. The CER standard deviation (0.18) and median (6.45%) are comparable to Phase 4A, but the aggregate CER is lower because the example-based constraint is more effective at preventing gratuitous rewrites.

The QALB transfer gap discussed in Section 4.6.3 is visible in the residual degradation. QALB examples are drawn from human typing errors, which follow a phonological-spelling distribution. OCR errors follow a visual-confusion distribution. The few-shot examples do not teach the model specifically to collapse repetition artefacts or to avoid dot-position changes; they teach it to correct the kinds of errors humans make when typing, which partially overlaps with but does not match the OCR error distribution.

Answer to RQ5: Few-shot examples from QALB improve correction relative to other augmentation conditions but still degrade performance by 75% CER relative to Phase 2. The transfer gap between QALB human typing errors and Qaari OCR visual-confusion errors limits the benefit of the QALB-derived demonstrations.

5.3.4 Phase 4C — CAMeL Morphological Post-Processing

Phase 4C achieves a CER of 5.77% and WER of 10.60% — *identical to Phase 2 in every metric*, including CER standard deviation (0.12) and median (3.28%). The revert strategy (Section 4.7.3) was triggered zero times across all 200 samples.

The zero-revert outcome follows directly from the morphological analysis of Phase 1 (Section 5.1.4): 100% of OCR errors are valid-but-wrong substitutions. The LLM's corrections are also morphologically valid Arabic forms, because Qwen3 is a fluent Arabic language model that rarely produces non-words. The revert condition — LLM word invalid AND OCR word valid — is therefore never satisfied.

Two interpretations of this result are possible. The first is that the CAMeL revert strategy is simply unnecessary for high-quality typewritten OCR: when the OCR errors are all valid words and the LLM corrections are all valid words, morphological validity provides no discriminative signal. The second interpretation is that Phase 4C's revert strategy addresses the wrong failure mode: it is designed to catch LLM hallucinations that produce non-word outputs, but the LLM on this dataset hallucinates *valid* alternative words, not non-words. Context-aware correction (requiring a language-modelling component, not just a morphological

oracle) would be needed to address this class of error.

The practical implication for Phase 6 is that adding CAMeL post-processing to any combination system is unlikely to produce improvement unless the combined prompt elicits a higher rate of morphological invalids from the LLM.

Answer to RQ6: CAMeL Tools morphological post-processing has no effect on correction quality in this experimental setting (0% CER change relative to Phase 2, zero reverts). OCR errors and LLM corrections are both morphologically valid Arabic forms; the revert condition is never satisfied.

5.3.5 Phase 4D — Self-Reflective Prompting

Phase 4D achieves a CER of 10.48% and WER of 16.12%, representing an 82% relative CER increase over Phase 2. The training-split error analysis (Section 4.8) produced accurate and interpretable statistics, reproduced here for reference:

Table 5.6: Phase 4D self-reflective insights derived from the PATS-A01 Akhbar training split. Fix rate: fraction of OCR errors of this type corrected by the LLM. Introduction rate: fraction of correct tokens of this type incorrectly modified by the LLM.

Error Type	Fix Rate	Introduction Rate
Insertion (repetition artefact)	97.35%	18.13%
Deletion	3.12%	212.50%
Other substitution	16.33%	163.27%
Overall	93.26%	25.06%

The training-split analysis reveals accurate and informative statistics. The LLM is exceptionally effective at correcting insertion errors (97.35% fix rate), which is consistent with the Phase 2 result for the repetition artefact. However, it is nearly unable to correct deletion errors (3.12% fix rate): reconstructing a missing character requires generating content that was not in the input, which the LLM rarely does correctly for single-character deletions in Arabic. The substitution fix rate of 16.33% is similarly low.

The error *introduction* rates are alarming. A 25.06% overall introduction rate means that for every four tokens that were correct in the OCR output, the LLM

incorrectly modifies one of them. For deletion errors (212.50%) and other substitutions (163.27%), the introduction rate exceeds 100%: the LLM introduces more new errors of these types than there were in the original OCR output.

These statistics were converted into Arabic-language diagnostic sentences and injected into the Phase 4D prompt using `build_self_reflective()`. The intent was to cause the model to apply corrections more cautiously in the domains where it performs poorly. The result shows this does not work: the CER increases by 82% relative to Phase 2.

The most likely explanation is that the diagnostic information carries a counterproductive framing. The injected text tells the model, in effect: "You tend to introduce errors in substitution and deletion categories." The model's instruction-following training interprets this as additional licence to intervene carefully — increasing, rather than decreasing, the number of positions it scrutinises. Paradoxically, the diagnostic warning that says "you over-correct" causes the model to over-correct more, not less.

The elevated CER standard deviation (0.24) and median (6.45%) confirm that self-reflective prompting causes more uniform degradation across the sample distribution than Phase 3 or Phase 5, consistent with the broad applicability of the diagnostic warnings.

Answer to RQ7: Self-reflective prompting — injecting the model's own diagnosed failure statistics into the prompt — does not improve correction. The 82% relative CER increase is consistent with the hypothesis that diagnostic warnings activate rather than suppress the model's intervention tendency.

5.3.6 Phase 5 — Retrieval-Augmented Generation

Phase 5 achieves a CER of 22.10% and WER of 26.93%, representing a 283% relative CER increase over Phase 2. This is the *worst result* across all experimental conditions and exceeds even the all-samples Phase 1 OCR baseline (17.01% CER).

The retrieval analysis reports an average top-1 cosine similarity of 0.805 between OCR queries and retrieved OpenITI passages. This high similarity score is an artefact of how inner-product similarity operates on sentence embeddings: short Arabic queries from a newspaper corpus and short passages from a classical religious corpus share surface-level morphological and lexical patterns (common Arabic words, definite article al-) that inflate embedding similarity without reflecting semantic or topical relatedness. The retrieved passages are not semantically relevant to the news sentences being corrected.

The model receives these classical Arabic passages as contextual grounding and uses them as stylistic templates. In doing so, it imports vocabulary, syntactic constructions, and even entire phrases from the retrieved classical text into the correction output, replacing modern newspaper vocabulary with archaic forms. This is the domain mismatch problem described in Section 4.9: OpenITI classical texts and PATS-A01 Akhbar news text are different registers of Arabic with substantially non-overlapping lexicons.

The CER standard deviation of 1.99 (second only to Phase 3's 2.19) confirms inconsistent, highly variable behaviour. When a retrieved passage happens to contain vocabulary similar to the input sentence, the model's intervention is particularly disruptive because the surface similarity masks the semantic gap. Some samples receive almost no retrieval-driven modification (low CER), while others are heavily rewritten (high CER).

Answer to RQ8: RAG grounded in the OpenITI classical Arabic corpus does not improve correction and produces the worst result (+283% CER relative to Phase 2). Domain mismatch between classical OpenITI text and modern newspaper content causes the model to import irrelevant vocabulary from retrieved passages. High embedding similarity scores (avg. 0.805) are misleading, reflecting surface-level lexical overlap rather than semantic relevance.

5.4 Phase 6 Results — Combinations and Ablation

Phase 6 tests 12 pre-selected combinations of knowledge augmentation strategies (Section 4.10) and performs component-level ablation on the full combined system. At the time of writing, Phase 6 inference has not yet been executed; this section reports the experimental design and defers quantitative results to future work.

The combinations to be evaluated are listed in Table 5.7. Each row identifies the combination by a short code built from the component initials: C (confusion matrix), R (rules), E (examples/few-shot), S (self-reflective), G (RAG/retrieval), and M (CAMeL post-processing). The selection rationale was described in Section 4.10: pairs were selected to combine complementary knowledge types; triples extend the best-performing pair; the full system combines all applicable components.

Table 5.7: Phase 6 combination designs. Results pending execution.

ID	Code	Components
6-01	C+E	Confusion matrix + few-shot examples
6-02	R+E	Rules + few-shot examples
6-03	C+R	Confusion matrix + rules
6-04	E+S	Few-shot + self-reflective
6-05	C+E+R	Confusion matrix + few-shot + rules
6-06	C+E+S	Confusion matrix + few-shot + self-reflective
6-07	R+E+S	Rules + few-shot + self-reflective
6-08	C+R+E+S	All prompt augmentations (no RAG)
6-09	C+R+E+S+G	Full prompt augmentation + RAG
6-10	C+E+M	Confusion matrix + few-shot + CAMeL
6-11	R+E+M	Rules + few-shot + CAMeL
6-12	C+R+E+S+G+M	Full system

The ablation design (Section 4.10) will report CER change when each component is removed from the full system (combination 6-12). A positive CER change upon component removal indicates that the component contributed positively to the combined system; a negative change indicates the component was harmful.

Based on the isolated phase results (Table 5.5), several predictions can be stated ahead of execution. First, Phase 4C (CAMeL) is unlikely to produce improvement in

any combination, since the revert condition was never triggered in isolation and the combined system is unlikely to produce more non-word outputs. Second, Phase 5 (RAG) is likely to remain harmful in combination, since the domain mismatch between OpenITI and Akhbar is a structural problem not resolved by combining with other strategies. Third, Phase 4B (few-shot, best isolated result) is the most promising component; combinations that include it are the most likely to approach or exceed the Phase 2 baseline.

RQ9 will be answered when Phase 6 results become available.

5.5 Summary and Ranking

Table 5.8 ranks all executed experimental conditions by CER, from best to worst.

Table 5.8: Final ranking of all executed experimental conditions by CER. Lower CER is better. Phase 6 results pending.

Rank	Phase	Method	CER	WER
1	1 (normal)	OCR Baseline (no LLM)	1.37%	5.04%
2	2 / 4C	Zero-Shot LLM / CAMeL	5.77%	10.60%
3	4B	Few-Shot (QALB)	10.09%	15.64%
4	4D	Self-Reflective	10.48%	16.12%
5	4A	Rule-Augmented	11.03%	17.70%
6	1 (all)	OCR Baseline (all)	17.01%	20.14%
7	3	Confusion Matrix	19.62%	24.35%
8	5	RAG (OpenITI)	22.10%	26.93%

The ranking reveals the core finding of this study: the *best result achievable with an LLM* (Phase 2, CER 5.77%) is still more than four times worse than the *best result achievable without an LLM* (Phase 1 normal, CER 1.37%). On high-quality typewritten Arabic text, the OCR engine is more accurate than the LLM corrector, and every knowledge augmentation strategy tested makes the corrector worse than the zero-shot baseline.

Table 5.9 summarises the answers to the nine research questions based on the results reported in this chapter.

Table 5.9: Summary of research question answers.

RQ	Finding	Verdict
RQ1	Qaari errors are systematic; dominated by repetition artefact (95.2%) and dot confusion on normal samples; all errors are valid-but-wrong.	Characterised
RQ2	Zero-shot LLM reduces aggregate CER by 66% by eliminating the repetition artefact, but increases CER on normal samples by 321%.	Partial
RQ3	Confusion matrix injection worsens CER by 240%.	Negative
RQ4	Rule augmentation worsens CER by 91%.	Negative
RQ5	Few-shot examples (QALB) worsens CER by 75%; best augmented result.	Negative
RQ6	CAMeL post-processing has zero effect; revert condition never triggered.	Neutral
RQ7	Self-reflective prompting worsens CER by 82%.	Negative
RQ8	RAG with OpenITI worsens CER by 283%; worst result overall.	Negative
RQ9	Phase 6 results pending.	Pending

The pattern of results is consistent and interpretable: every form of additional knowledge injected into the prompt increases the model's propensity to intervene on the input text, and on an input text that is already 98.63% accurate (normal-samples CER 1.37%), intervention almost always introduces errors. The over-correction problem, quantified at a 25.06% error introduction rate in Phase 4D, is not a failure of any individual augmentation strategy — it is a property of the LLM's instruction-following behaviour on high-quality input. Chapter 6.7 analyses this finding in depth and considers the conditions under which knowledge augmentation would produce a different outcome.

Chapter 6

Discussion and Conclusion

This chapter interprets the findings reported in Chapter 5.5, situates them in the broader literature, acknowledges the limitations of the study honestly, and charts future directions. Section 6.1 revisits each research question. Section 6.2 develops the central unexpected finding — the over-correction problem — as a cohesive theoretical and empirical argument. Section 6.3 considers the conditions under which the findings would likely reverse. Section 6.4 offers practical system design recommendations. Section 6.5 enumerates limitations. Section 6.6 outlines future research directions. Section 6.7 provides the concluding statement.

6.1 Revisiting the Research Questions

RQ1: OCR Error Characterisation

Question: What are the characteristics of errors produced by Qaari on Arabic typewritten text? What error types dominate, and how severe is the overall degradation?

Finding: Qaari's primary failure mode on the PATS-A01 Akhbar font is a catastrophic repetition artefact that affects 0.5% of samples and accounts for 95.2% of all character errors (CER 17.01% overall, 1.37% on normal samples). On normal samples, the dominant error class is dot confusion (ظ/ط, ب/ي, ق/ف and related pairs), and all OCR errors produce morphologically valid Arabic forms.

Significance: The high OCR quality on normal samples (CER 1.37%) places this dataset in the regime where post-OCR correction is most risky. When fewer than 2% of characters are erroneous, a corrector that cannot reliably identify which characters are wrong will corrupt more correct characters than it fixes. This dataset is therefore not a favourable context for LLM-based correction, and the subsequent results must be interpreted in that light.

Caveat: The Akhbar font is a well-rendered typewritten font with unambiguous glyph shapes. Other PATS-A01 fonts or handwritten text (KHATT) may exhibit higher error rates and different error distributions, which could produce qualitatively different results for the correction phases.

RQ2: Zero-Shot Correction Capability

Question: Can a zero-shot instruction-tuned LLM correct Arabic OCR errors without any task-specific guidance?

Finding: Yes, partially. The zero-shot LLM reduces aggregate CER from 17.01% to 5.77% (66% relative reduction) by collapsing repetition artefacts effectively. On normal samples, however, it increases CER from 1.37% to 5.77% (321% relative increase), demonstrating systematic over-correction.

Significance: This duality — effective on catastrophic failures, harmful on high-quality input — suggests that zero-shot LLM correction should be applied *selectively*, not universally. It is a reliable safety net for degenerate OCR output, but a liability for good OCR output. The finding aligns with Kang et al. [32] who observe that LLMs show strong performance on heavily corrupted text but introduce errors on clean text.

Caveat: The zero-shot prompt used was the aggressive version (v1). The conservative version (v2) might achieve a better balance between fixing artefacts and preserving correct text, though the Phase 4D error introduction rate of 25.06% suggests the over-correction tendency is deep-seated and unlikely to be fully resolved by prompt rewording alone.

RQ3: Confusion Matrix Injection

Question: Does injecting OCR-specific error knowledge (confusion matrix) into the prompt improve correction beyond zero-shot?

Finding: No. Confusion matrix injection increases CER by 240% relative to Phase 2 (19.62% vs. 5.77%), producing the second-worst result overall.

Significance: Injecting a list of characters the model should scrutinise increases, not decreases, its intervention rate on those characters. This is a concrete demonstration of the prompt-as-activation-signal problem: informing the model that certain confusions exist activates its correction reflex on those characters, regardless of whether those characters are actually confused in the input. The model cannot distinguish between "the confusion matrix says this pair is confusable" and "this specific character in this specific token is confused."

RQ4: Rule-Augmented Prompting

Question: Does injecting explicit Arabic orthographic rules improve correction?

Finding: No. Rule augmentation increases CER by 91% relative to Phase 2 (11.03% vs. 5.77%).

Significance: Prescriptive orthographic rules apply to all text, not just to OCR errors. The model applies them to correctly rendered tokens, producing over-normalisation. This is consistent with the known trade-off between generality and precision in linguistic rule injection: general rules cover many phenomena but also fire on correct text.

RQ5: Few-Shot Examples

Question: Does providing error-correction examples from the QALB corpus improve correction?

Finding: No, but Phase 4B is the best augmented result. CER increases by 75% relative to Phase 2 (10.09% vs. 5.77%), the smallest degradation among active augmentation conditions.

Significance: Example-based guidance is more effective than abstract rules or character lists at constraining the model's behaviour. Showing specific input–output pairs implicitly communicates the expected scale of correction, reducing the gratuitous rewriting that characterises Phases 3, 4A, and 5. The residual degradation reflects the QALB transfer gap: human typing errors and OCR visual-confusion errors are different distributions, and the QALB examples do not teach the model how to handle repetition artefacts, dot-position swaps, or the conservative posture required for high-quality input.

RQ6: CAMeL Morphological Post-Processing

Question: Does morphological post-processing via CAMeL Tools improve the quality of LLM-corrected text?

Finding: No, and it has zero effect (0% CER change; zero reverts on 200 samples).

Significance: The revert strategy addresses a failure mode — LLM-introduced morphological invalids — that does not occur in this dataset. When both OCR errors and LLM corrections are morphologically valid Arabic forms (as confirmed by Phase 1 morphological analysis), the revert condition is never satisfied. This result clarifies the scope of morphological post-processing: it is useful only when the LLM produces non-word outputs, which is a symptom of a poorly calibrated or insufficiently fluent model. Qwen3-4B, as a strong multilingual model, almost never produces non-words in Arabic.

RQ7: Self-Reflective Prompting

Question: Does self-reflective prompting — injecting the model's own diagnosed failure patterns — improve correction?

Finding: No. Self-reflective prompting increases CER by 82% relative to Phase 2 (10.48% vs. 5.77%).

Significance: The diagnostic statistics derived from training-split analysis are accurate (97.35% insertion fix rate, 25.06% overall introduction rate), but injecting them into the prompt does not translate the accurate diagnosis into improved behaviour. The model's instruction-following training interprets the diagnostic warnings as additional licence to intervene, not as constraints to hold back. This finding has implications beyond OCR correction: it suggests that self-diagnostic prompting strategies [24] may be less effective when the model's primary failure mode is over-intervention rather than under-intervention. Reflexion-style feedback was designed for tasks where the model is not trying hard enough; in this setting, the model is trying too hard.

RQ8: Retrieval-Augmented Generation

Question: Does retrieval-augmented generation grounded in the OpenITI corpus improve correction?

Finding: No, and it produces the worst result overall. RAG increases CER by 283% relative to Phase 2 (22.10% vs. 5.77%), exceeding even the all-samples OCR baseline (17.01%).

Significance: Domain mismatch between the classical OpenITI corpus and modern Akhbar newspaper text is fatal to the RAG strategy in this setting. Retrieved passages are used by the model as stylistic templates, importing archaic vocabulary and constructions that do not belong in the target text. The misleadingly high embedding similarity scores (average 0.805) illustrate a known weakness of general-purpose embedding models: they compute similarity based on shared surface tokens

(common Arabic words, the definite article) rather than topical or semantic relevance.

RQ9: Best Combination (Phase 6)

Question: Which combination of knowledge augmentation strategies produces the best correction, and what does each component individually contribute?

Finding: Phase 6 results are pending. Based on the isolated results, combinations that include few-shot examples (the best isolated augmentation) and exclude RAG (the worst) are the most likely candidates for improvement. Whether any combination can exceed the zero-shot Phase 2 baseline remains an open question.

6.2 The Over-Correction Problem

The central unexpected finding of this study is that *every active knowledge augmentation strategy tested degrades correction quality relative to zero-shot*, and that the shared mechanism of degradation is *over-correction*: the LLM modifies tokens that are already correct. This section develops a unified account of the phenomenon.

6.2.1 What Over-Correction Is

Over-correction refers to the LLM changing tokens in the OCR output that correspond to correct characters in the ground truth. On a corpus where the normal-samples OCR CER is 1.37%, the expected number of erroneous characters per sentence is very low. Any intervention on a correct character introduces a new error that did not exist before correction.

The Phase 4D training-split analysis provides a precise quantification: Qwen3-4B-Instruct-2507 has an overall error introduction rate of 25.06%. This means that, on average, for every four characters that were correctly recognised by Qaari, the

LLM incorrectly modifies one of them. For specific error categories, the introduction rates are even more severe: 163.27% for substitution errors (the LLM introduces 1.63 new substitution errors for every substitution it fixes) and 212.50% for deletion errors (the LLM introduces 2.13 new deletion errors for every deletion it fixes). The only category where the LLM excels is insertion errors (repetition artefacts), where it achieves a 97.35% fix rate with only an 18.13% introduction rate.

The practical consequence is clear. Given that Qaari on the Akhbar font makes very few errors on normal text, the LLM's 25.06% introduction rate far outweighs the small number of errors it fixes. Correction makes things worse for the typical sample.

6.2.2 Why Over-Correction Happens

The mechanism is rooted in instruction-tuning. A language model trained to follow the instruction "correct this text" has been trained on data where the correct action is to produce a modified output. The training signal rewards modification: responses that return the input unchanged score poorly against a correction reward, while responses that make changes — even small, unnecessary ones — are more consistent with the training distribution.

The conservative instruction appended to all prompts in this study — "if the text is already correct, return it unchanged" — partially mitigates this tendency but cannot fully counteract it. The model has a strong prior toward intervention, and a single sentence in the system prompt cannot override thousands of training examples that demonstrate correction as the expected behaviour.

This interpretation is consistent with the psychology of LLM generation: the model samples from a conditional distribution $P(\text{output} \mid \text{instruction}, \text{input})$ where the instruction primes it toward corrective outputs. Arabic words that could plausibly have alternative forms (e.g., a word with a hamza whose placement could vary) will be corrected even when the original form is already the appropriate one.

6.2.3 Why Augmentation Makes Over-Correction Worse

Every knowledge augmentation element added to the prompt signals to the model that the input text is problematic and that active correction is warranted. Each element acts as an *activation signal* for the model's correction behaviour:

- A confusion list says: "the OCR engine confuses these characters — be suspicious of these characters in the input." The model responds by becoming suspicious of those characters and changing them, even when they are correct.
- Orthographic rules say: "apply these rules to the text." The model applies them universally, including to correctly rendered tokens.
- Few-shot examples say: "here is what correction looks like." The model attempts to produce output at a similar edit distance to the examples, even when the input is already correct.
- Self-reflective diagnostics say: "you tend to make these errors." The model interprets this as evidence that the input is likely to contain errors and intervenes more.
- Retrieved passages provide alternative text that the model implicitly treats as a corrective reference, pulling its output toward the retrieved content.

The cumulative effect of adding more augmentation elements is a monotonic increase in the model's intervention rate, which translates directly into a monotonic increase in CER on high-quality input. The ranking in Table 5.5 (Chapter 5.5) is consistent with this account: Phase 4B (least activating augmentation, 75% degradation) is the best; Phase 5 (most foreign reference text injected, 283% degradation) is the worst.

6.2.4 Connection to the Broader Literature

Over-correction in LLMs is not unique to OCR correction. Analogous effects have been observed in related settings.

In *fact verification*, language models provided with retrieved passages sometimes introduce hallucinations that align with the retrieved content rather than the ground truth [21]. The model treats the retrieved content as authoritative and modifies its output accordingly — a direct parallel to the Phase 5 RAG behaviour observed here.

In *chain-of-thought prompting* [20], models that reason step-by-step sometimes arrive at wrong answers that they would have answered correctly with a direct response. Extended reasoning gives the model more opportunities to drift from the correct answer. The self-reflective prompting in Phase 4D is analogous: the model's extended reflection on its failure patterns gives it more opportunities to introduce unsolicited corrections.

More broadly, the finding that *simpler prompts outperform augmented prompts* on this task runs counter to the prevailing assumption in prompt engineering research that more information is always better. It suggests that the optimal augmentation level depends on the error density of the input: high-error-density input benefits from more guidance; low-error-density input is harmed by it.

6.3 When Would Augmentation Help?

The findings of this study are specific to a well-defined experimental context: a high-quality typewritten corpus, a single OCR engine, a single 4-billion-parameter LLM, and a single document domain. The results would likely differ under the following conditions.

6.3.0.0.1 Higher baseline CER. If the OCR output had a baseline CER of 20–30% (as might be expected on difficult fonts, degraded documents, or handwritten text such as KHATT), there would be proportionally more errors to fix and fewer correct tokens to corrupt. The LLM's 25.06% introduction rate would still hold, but it would represent a smaller fraction of the total token count. In that regime, the net effect of correction could be positive, and knowledge augmentation that further increases the intervention rate might produce net benefit. Evaluating on KHATT is

the highest-priority next step for this reason.

6.3.0.0.2 Domain-matched retrieval corpus. Phase 5 is a worst-case RAG evaluation: OpenITI classical Arabic against PATS-A01 modern Arabic newspaper text. Building a retrieval corpus from modern Arabic news articles (or, ideally, from the same news agency as the Akhbar font) would remove the domain mismatch and allow the retrieval to surface genuinely relevant passages. Under these conditions, RAG could provide contextual grounding that helps the model distinguish between alternative word forms rather than providing misleading stylistic templates.

6.3.0.0.3 OCR-native few-shot pairs. Phase 4B uses QALB pairs derived from human typing errors. The transfer gap to OCR errors is imperfect. Generating few-shot pairs by running Qaari on clean ground-truth text, pairing the OCR output with the original, and selecting pairs that illustrate dot-confusion corrections and repetition collapses would provide examples that directly target the OCR error distribution. This is straightforward to implement and is expected to significantly improve Phase 4B outcomes.

6.3.0.0.4 Larger or fine-tuned models. Qwen3-4B is a relatively compact model. Larger models (Qwen3-14B, GPT-4) may have a stronger implicit prior toward conservatism, having been exposed to more text and more instruction-following training that includes explicit "do not change what is correct" examples. Alternatively, fine-tuning a model specifically for OCR correction, with training examples that reward returning correct text unchanged, would directly address the over-correction tendency. Such a model would have a different intervention prior and might benefit substantially from knowledge augmentation.

6.3.0.0.5 Confidence-based selective correction. Rather than applying LLM correction to all samples uniformly, a selective strategy could route only those samples likely to contain errors to the corrector. OCR engines often produce character-level confidence scores; samples below a confidence threshold could be flagged for

correction while high-confidence outputs are left untouched. This would limit the LLM's exposure to correct text and substantially reduce the over-correction rate.

6.4 Implications for System Design

The following practical recommendations are derived from the experimental findings.

6.4.0.0.1 For high-quality typewritten Arabic OCR: use zero-shot, do not augment. On input where the baseline CER is below 5%, adding knowledge augmentation to the LLM prompt is actively harmful. The zero-shot corrector should be preferred over any augmented variant. The conservative instruction ("return unchanged if correct") should be included, though it provides only partial protection against over-correction.

6.4.0.0.2 Apply LLM correction selectively, not universally. The LLM is reliable for correcting catastrophic OCR failures (repetition artefacts, severe degradation) but harmful for high-quality output. A detection step — either using OCR confidence scores or a simple length-ratio heuristic (LLM output significantly shorter than OCR output signals repetition collapse) — should precede correction, routing only high-error samples to the LLM.

6.4.0.0.3 The LLM is a safety net for runaway artefacts. The 97.35% insertion fix rate demonstrates that zero-shot LLM correction is a highly effective remedy for the Qaari repetition bug. Deploying the LLM specifically as an artefact detector for samples whose OCR output length exceeds some multiple of the expected text length (e.g., 5) would deliver the primary benefit of Phase 2 at a fraction of the inference cost.

6.4.0.0.4 Include CAMeL post-processing as a zero-cost defensive layer. Phase 4C has zero effect in this setting but also zero downside. It adds negligible computational cost (a morphological lookup per token, cached with an LRU cache). For settings with a less fluent LLM, or with more morphologically complex error types, it could provide benefit. Its inclusion is recommended as a defensive measure that cannot degrade results.

6.4.0.0.5 Do not deploy RAG unless the retrieval corpus is domain–matched. Phase 5 demonstrates that a domain-mismatched retrieval corpus is actively harmful, producing the worst result across all conditions. The retrieval corpus must be drawn from the same domain, register, and time period as the target documents. Before deploying RAG in a production Arabic OCR pipeline, retrieval precision should be validated on held-out samples and a CER threshold should be applied to confirm the corpus is helping, not hurting.

6.5 Limitations

This study acknowledges the following limitations.

6.5.0.0.1 Single dataset. All quantitative results are derived from 200 samples of the PATS-A01 Akhbar font, training split. This is a single typewritten Arabic font from a specific newspaper corpus. Results may not generalise to other PATS-A01 fonts (some of which have higher OCR error rates), to handwritten text (KHATT), or to other document domains (historical manuscripts, printed books, signage).

6.5.0.0.2 Single OCR engine. Qaari's error characteristics — particularly the repetition artefact and the preponderance of dot-confusion substitutions — shape all downstream results. An OCR engine with different failure modes (e.g., Tesseract, TrOCR, or commercial engines) would present a different error distribution to the LLM. The confusion matrix injection (Phase 3), for instance, might behave differently

if the confusion matrix reflected a different set of character pairs.

6.5.0.0.3 Single LLM. Qwen3-4B-Instruct-2507 is the sole model evaluated. The model's size (4B parameters), training data (multilingual instruction tuning), and inference configuration (greedy decoding, no thinking mode) are specific choices that may not represent the behaviour of other instruction-tuned models. Larger models may over-correct less; Arabic-specific models may handle dot-confusion errors differently.

6.5.0.0.4 No statistical significance testing on isolated phases. Phases 2–5 are each evaluated on the same 200 samples with no held-out test set. The differences between phases (e.g., Phase 2 CER 5.77% vs. Phase 4B CER 10.09%) are reported without paired statistical tests. Phase 6 is designed to include paired t-tests and bootstrap confidence intervals, but the isolated phase results are presented as descriptive statistics only.

6.5.0.0.5 Phase 6 not yet executed. The combination and ablation study (Phase 6, answering RQ9) is designed but not yet executed. The most important research question — which combination of knowledge sources produces the best result — remains unanswered. The discussion in Section 6.1 (RQ9) is therefore speculative.

6.5.0.0.6 QALB transfer gap. Phase 4B few-shot examples are drawn from the QALB corpus, which contains human typing errors rather than OCR visual-confusion errors. The transfer gap between these two distributions limits the ecological validity of the few-shot condition. The Phase 4B result should be interpreted as a lower bound on what few-shot correction could achieve with OCR-native examples.

6.5.0.0.7 Classical corpus for RAG. Phase 5 uses OpenITI, a corpus of classical and historical Arabic texts. This is a worst-case domain match for the modern Akhbar newspaper corpus. The 283% CER increase should not be interpreted

as evidence that RAG cannot help Arabic OCR correction in general; it should be interpreted as evidence that a severely domain-mismatched retrieval corpus causes catastrophic degradation.

6.5.0.0.8 Diacritic stripping in evaluation. All CER and WER figures are computed after stripping diacritics from both the reference and hypothesis texts (Section 3.3.1). This is the standard evaluation protocol for Arabic NLP, as diacritics are optional in most printed Arabic. However, if the application requires diacritics (e.g., Quranic text, children's educational material), the evaluation protocol and the correction task would be more demanding.

6.6 Future Work

6.6.0.0.1 Evaluate on KHATT and all PATS-A01 fonts. The highest-priority next step is expanding evaluation to the 199 KHATT handwritten samples and all 8 PATS-A01 fonts. Handwritten text is expected to exhibit higher baseline CER, which may reverse the over-correction dynamic and produce positive results for knowledge augmentation. Evaluating across all 8 fonts would also reveal whether the Akhbar results generalise to other typewritten Arabic styles.

6.6.0.0.2 Generate OCR-native few-shot pairs for Phase 4B. Running Qaari on a clean ground-truth corpus, pairing OCR output with the original text, and selecting pairs that exhibit dot-confusion corrections and repetition collapses would produce few-shot examples that directly match the OCR error distribution. This experiment requires no GPU inference (Qaari is a local engine) and could substantially improve Phase 4B outcomes.

6.6.0.0.3 Build a domain-matched retrieval corpus for Phase 5. Replacing OpenITI with a modern Arabic newspaper corpus would remove the classical–modern domain mismatch. A corpus assembled from publicly available Arabic news sources

(Aljazeera, BBC Arabic, Al-Ahram) in the Akhbar style would provide semantically relevant retrieval candidates. The Phase 5 experimental infrastructure (FAISS index, retrieval pipeline) requires no modification; only the source corpus changes.

6.6.0.0.4 Fine-tune Qwen3-4B for conservative OCR correction. Training a model on OCR correction pairs with explicit "return unchanged" examples for high-quality inputs would directly address the over-correction problem. The training objective would reward the model for returning correct text unchanged (and penalise unsolicited modifications), which would change the model's intervention prior without requiring prompt engineering. Such a fine-tuned model would be a stronger base for knowledge augmentation.

6.6.0.0.5 Test with larger models. Evaluating Qwen3-14B, GPT-4o, or Claude Sonnet on the same experimental conditions would assess whether the over-correction tendency diminishes with scale. Larger models have been exposed to more instruction-following data and may have a stronger implicit conservatism. This experiment requires no changes to the pipeline — only the model identifier in the configuration file.

6.6.0.0.6 Study over-correction as a standalone research question. The 25.06% error introduction rate observed in Phase 4D is a striking finding that warrants dedicated investigation. Characterising the types of tokens most likely to be over-corrected (frequency, morphological complexity, ambiguity of alternative forms), and developing a calibrated refusal mechanism that the model can apply at the token level, would be a valuable contribution to the LLM reliability literature.

6.6.0.0.7 Confidence-guided selective correction. Combining OCR confidence scores with LLM correction decisions would limit the corrector's exposure to high-confidence (likely correct) OCR output. A selective routing system that only sends low-confidence samples or samples above a length-ratio threshold to the LLM would deliver the primary correction benefit while avoiding the majority of

over-correction cases.

6.6.0.0.8 Extension to dialectal Arabic OCR. Egyptian, Gulf, and Levantine dialects present additional OCR challenges (non-standard spelling, code-switching, absent diacritics) and limited NLP tool support. Extending the experimental framework to dialectal Arabic would require new ground-truth corpora and would test whether the over-correction problem is specific to MSA or applies more broadly.

6.7 Conclusion

This thesis set out to answer a single central question: *Can a lightweight open-source LLM correct Arabic OCR errors through prompt engineering alone, and does adding more linguistic or retrieval-based knowledge to the prompt improve the correction?*

The answer is nuanced. On the first part of the question: yes, a zero-shot LLM can improve aggregate OCR quality, primarily by eliminating catastrophic repetition artefacts that inflate CER. A 66% aggregate CER reduction (17.01% to 5.77%) is a substantial improvement on an all-samples basis. On the second part of the question: no, knowledge augmentation does not improve upon zero-shot correction. Every augmentation strategy tested — confusion matrix injection, orthographic rules, few-shot examples, self-reflective diagnostics, and retrieval-augmented generation — produced higher CER than the zero-shot baseline, with degradations ranging from 75% to 283%.

The underlying explanation is the *over-correction problem*: on high-quality typewritten Arabic text where the OCR error rate is 1.37%, the LLM's 25.06% error introduction rate means that correction is net harmful for the average sample. Adding knowledge elements to the prompt increases the LLM's intervention rate, which worsens an already unfavourable error-introduction-to-error-fix ratio.

The practical implication is direct: for high-quality typewritten Arabic OCR,

the simplest strategy is the best. Zero-shot correction should be applied selectively — only to samples suspected of containing errors, identified by confidence scores or output-length anomalies — and should not be augmented with additional knowledge that signals to the model that the text is unreliable.

The broader significance of this result extends beyond Arabic OCR. Negative results in prompt engineering research are informative precisely because they identify failure modes that successful systems must guard against. The finding that more information in the prompt is not always better — and that on high-quality input, it is consistently worse — is a contribution to the collective understanding of how instruction-tuned LLMs behave when tasked with selective modification. As LLMs are deployed in increasingly diverse text-processing pipelines, understanding the conditions under which they over-intervene is as important as understanding the conditions under which they succeed.

The study of Arabic post-OCR correction is not finished. The KHATT handwritten dataset, larger models, domain-matched retrieval corpora, and OCR-native few-shot pairs all represent directions where the findings of this thesis would likely look different. Future work that pursues these directions will build on the experimental framework, the three-stage pipeline, and the negative results documented here to develop a more complete picture of when and how LLMs can be trusted to improve imperfect Arabic text.

References

[1] T. Brown et al., "Language models are few-shot learners," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 1877–1901, 2020.

[2] X. Tong and D. A. Evans, "A statistical approach to automatic OCR error correction in context," in *Proceedings of the Fourth Workshop on Very Large Corpora*, 1996, pp. 88–100.

[3] C. Holes, *Modern Arabic: Structures, Functions, and Varieties*. Washington, D.C.: Georgetown University Press, 2004.

[4] S. Mori, C. Y. Suen, and K. Yamamoto, "Historical review of OCR research and development," *Proceedings of the IEEE*, vol. 80, no. 7, pp. 1029–1058, 1992.

[5] A. Graves, S. Fernández, F. Gomez, and J. Schmidhuber, "Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks," *Proceedings of the 23rd International Conference on Machine Learning (ICML)*, pp. 369–376, 2006.

[6] R. Smith, "An overview of the tesseract OCR engine," in *Proceedings of the Ninth International Conference on Document Analysis and Recognition (ICDAR)*, IEEE, 2007, pp. 629–633.

[7] M. Li et al., "TrOCR: Transformer-based optical character recognition with pre-trained models," *arXiv preprint arXiv:2109.10282*, 2021.

[8] G. Kim et al., "OCR-free document understanding transformer," in *Proceedings of the European Conference on Computer Vision (ECCV)*, Springer, 2022, pp. 498–517.

[9] C. Rigaud, A. Doucet, M. Coustaty, and J.-P. Moreux, "ICDAR 2019 competition on post-OCR text correction," *Proceedings of the International Conference on Document Analysis and Recognition (ICDAR)*, pp. 1588–1593, 2019.

[10] Y. Bengio, R. Ducharme, P. Vincent, and C. Jauvin, "A neural probabilistic language model," *Journal of Machine Learning Research (JMLR)*, vol. 3, pp. 1137–1155, 2003.

[11] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[12] A. Vaswani et al., "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[13] L. Ouyang et al., "Training language models to follow instructions with human feedback," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, pp. 27 730–27 744, 2022.

[14] Qwen Team, "Qwen3 technical report," *arXiv preprint arXiv:2505.09388*, 2025.

[15] W. Antoun, F. Baly, and H. Hajj, "AraBERT: Transformer-based model for arabic language understanding," in *Proceedings of the LREC Workshop on Language Models and their Applications*, 2020, pp. 9–15.

[16] G. Inoue, B. Alhafni, N. Baimukan, H. Bouamor, and N. Habash, "CAMeL-BERT: A collection of pre-trained models for arabic NLP," in *Proceedings of the Sixth Arabic Natural Language Processing Workshop (WANLP)*, 2021, pp. 194–207.

[17] W. Antoun, F. Baly, and H. Hajj, "AraGPT2: Pre-trained transformer for arabic language generation," *Proceedings of the Sixth Arabic Natural Language Processing Workshop (WANLP)*, pp. 196–207, 2021.

[18] E. M. B. Nagoudi, A. Elmadany, and M. Abdul-Mageed, "AraT5: Text-to-text transformers for arabic language understanding and generation," *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 628–647, 2022.

[19] P. Liu, W. Yuan, J. Fu, Z. Jiang, H. Hayashi, and G. Neubig, "Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing," *ACM Computing Surveys*, vol. 55, no. 9, pp. 1–35, 2023.

[20] J. Wei et al., "Chain-of-thought prompting elicits reasoning in large language models," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, pp. 24 824–24 837, 2022.

[21] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.

[22] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," 3, vol. 7, IEEE, 2021, pp. 535–547.

[23] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 3982–3992, 2019.

[24] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.

[25] B. Mohit, A. Rozovskaya, N. Habash, W. Zaghouani, and O. Obeid, "The first QALB shared task on automatic text correction for arabic," in *Proceedings of the EMNLP Workshop on Arabic Natural Language Processing*, 2014, pp. 39–47.

[26] W. Zaghouani et al., "Large scale arabic error annotation: Guidelines and framework," in *Proceedings of the Ninth International Conference on Language Resources and Evaluation (LREC)*, 2014, pp. 2362–2369.

[27] L. Nigst, M. Romanov, S. B. Savant, M. Seydi, and P. Verkinderen, "OpenITI: A machine-readable corpus of islamicate texts," *Zenodo*, 2020, Version 2020.1.2.

[28] M. Romanov, "Algorithmic analysis of medieval arabic biographical collections," *Speculum*, vol. 92, no. S1, S226–S246, 2017.

[29] O. Obeid et al., "CAMeL tools: An open source python toolkit for arabic natural language processing," in *Proceedings of the Twelfth Language Resources and Evaluation Conference (LREC)*, 2020, pp. 7368–7378.

[30] O. Kolak and P. Resnik, "OCR error correction using a noisy channel model," in *Proceedings of the Human Language Technology Conference and Conference on Empirical Methods in Natural Language Processing (HLT-EMNLP)*, 2005, pp. 257–264.

[31] R. Dong and D. Smith, "Multi-input attention for unsupervised OCR correction," in *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2018, pp. 2363–2372.

[32] L. Kang et al., "Large language models are better at OCR error correction than you think," *arXiv preprint*, 2024.

[33] S. A. Mahmoud et al., "KHATT: An open arabic offline handwritten text database," *Pattern Recognition*, vol. 47, no. 3, pp. 1096–1112, 2014.

Appendix A

Full Prompt Templates

This appendix contains the complete system prompt templates used in each experimental phase. All prompts are written in Arabic, as described in Section 3.6. English translations are provided for accessibility. The user message in every phase is the raw OCR text to be corrected; only the system messages differ across phases.

A.1 Phase 2 — Zero-Shot Prompt (v1, Aggressive)

Version tag: `p2v1` **Source:** `PromptBuilder.ZERO_SHOT_SYSTEM_V1`

مهمتك تصحيح أخطاء التعرف أنت مصحح نصوص عربية متخصص.
أعد النص المصحح فقط بدون أي الضوئي (صرض) في النص العربي.
شرح أو تعليق إضافي.

English translation:

You are a specialised Arabic text corrector. Your task is to correct optical character recognition (OCR) errors in Arabic text. Return only the corrected text with no explanation or additional commentary.

A.2 Phase 2 — Zero-Shot Prompt (v2, Conservative)

Version tag: `p2v2` **Source:** `PromptBuilder.ZERO_SHOT_SYSTEM_V2`

النص التالي ناتج عن نظام التعرف الضوئي أنت مدقق نصوص عربية.
مهمتك: إذا وقد يكون صحيحاً بالكامل أو يحتوي على أخطاء قليلة.
إذا وجدت خطأ واضحاً فقط، كان النص صحيحاً، أعده كما هو بالضبط.
أعد النص فقط بدون أي شرح. صححه. لا تغير ما هو صحيح.

English translation:

You are an Arabic text proofreader. The following text is the output of an OCR system and may be completely correct or may contain a few errors. Your task: if the text is correct, return it exactly as is. If you find only a clear error, correct it. Do not change what is correct. Return only the text with no explanation.

A.3 Phase 3 — OCR-Aware Prompt (p3v1)

Version tag: `p3v1` **Source:** `PromptBuilder.OCR_AWARE_SYSTEM_V1`

The placeholder `{confusion_context}` is replaced at runtime with the top-10 confusion pairs formatted by `ConfusionMatrixLoader`.

أنت مصحح نصوص عربية متخصص في تصحيح مخرجات نظام صَرِ
فيما يلي أبرز الأخطاء الشائعة التي يرتكبها هذا النظام: للتعرف الضوئي.

{confusion_context}

أعد النص صحح النص التالي مع الانتباه بشكل خاص لهذه الأخطاء.
المصحح فقط بدون أي شرح أو تعليق إضافي.

English translation:

You are an Arabic text corrector specialised in correcting the output of the Qaari optical character recognition system. The following are the most common errors made by this system:

{confusion_context}

Correct the following text paying special attention to these errors. Return only the corrected text with no explanation or additional commentary.

A.4 Phase 4A — Rule-Augmented Prompt (p4av1)

Version tag: `p4av1` **Source:** `PromptBuilder.RULES_SYSTEM_V1`

The placeholder `{rules_context}` is replaced at runtime with the formatted Arabic orthographic rules from `RulesLoader`.

راعِ القواعد الإملائية التالية عند أنت مصحح نصوص عربية متخصص.
تصحيح النص:

{rules_context}

أعد النص صحح النص التالي مع الانتباه بشكل خاص لهذه القواعد.
المصحح فقط بدون أي شرح أو تعليق إضافي.

English translation:

You are a specialised Arabic text corrector. Apply the following orthographic rules when correcting the text:

{rules_context}

Correct the following text paying special attention to these rules. Return only the corrected text with no explanation or additional commentary.

A.5 Phase 4B — Few-Shot Prompt (p4bv1)

Version tag: `p4bv1` **Source:** `PromptBuilder.FEWSHOT_SYSTEM_V1`

The placeholder `{examples_context}` is replaced at runtime with 5 QALB correction pairs formatted by `QALBLoader`.

فيما يلي أمثلة على تصحيح أخطاء أنت مصحح نصوص عربية متخصص.
الكتابة العربية:

{examples_context}

أعد النص المصحح فقط بدون أي صحح النص التالي بنفس الأسلوب.
شرح أو تعليق إضافي.

English translation:

You are a specialised Arabic text corrector. The following are examples of correcting Arabic writing errors:

{examples_context}

Correct the following text in the same style. Return only the corrected text with no explanation or additional commentary.

A.6 Phase 4D — Self-Reflective Prompt (p4dv1)

Version tag: `p4dv1` **Source:** `PromptBuilder.SELF_REFLECTIVE_SYSTEM_V1`

The placeholder `{insights_context}` is replaced at runtime with the Arabic diagnostic text from `LLMInsightsLoader` (see Appendix B for the full generated text).

أنت مصحح نصوص عربية متخصص. بناءً على تحليل أخطائك السابقة
مشابهة، إليك ملاحظات مهمة لتحسين في تصحيح نصوص عربية
أدائك:

{insights_context}

أعد النص المصحح فقط صحح النص التالي مع مراعاة هذه الملاحظات.
بدون أي شرح أو تعليق إضافي.

English translation:

You are a specialised Arabic text corrector. Based on an analysis of your previous errors correcting similar Arabic texts, here are important notes for improving your performance:

{insights_context}

Correct the following text taking these notes into account. Return only the corrected text with no explanation or additional commentary.

A.7 Phase 5 — RAG Prompt (p5v1)

Version tag: `p5v1` **Source:** `PromptBuilder.RAG_SYSTEM_V1`

The placeholder `{retrieval_context}` is replaced at runtime with the top-3 sentences retrieved from the OpenITI FAISS index.

فيما يلي نصوص عربية صحيحة أنت مصحح نصوص عربية متخصص.
مشابهة للنص المراد تصحيحه، استخدمها كمرجع:

{retrieval_context}

أعد النص المصحح صحح النص التالي مستعيناً بهذه النصوص المرجعية.
فقط بدون أي شرح أو تعليق إضافي.

English translation:

You are a specialised Arabic text corrector. The following are correct Arabic texts similar to the text to be corrected; use them as a reference:

{retrieval_context}

Correct the following text using these reference texts as a guide. Return only the corrected text with no explanation or additional commentary.

A.8 Phase 6 — Combined Prompt (p6v1)

Version tag: `p6v1` **Source:** `PromptBuilder.build_combined()`

The combined prompt assembles only non-empty context sections in a fixed order: (1) confusion context, (2) rules context, (3) examples context, (4) retrieval context, (5) self-reflective insights. Below is the full template with all five sections present.

أنت مصحح نصوص عربية متخصص. استخدم المعلومات التالية لتصحيح أخطاء التعرف الضوئي:

أولاً: أخطاء شائعة في هذا النظام: *{confusion_context}*

ثانياً: قواعد إملائية مهمة: *{rules_context}*

ثالثاً: أمثلة على التصحيح: *{examples_context}*

رابعاً: نصوص مرجعية صحيحة مشابهة: *{retrieval_context}*

خامساً: ملاحظات تحسين الأداء الذاتي: *{insights_context}*

صحح النص التالي. أعد النص المصحح فقط بدون أي شرح أو تعليق.

English translation:

You are a specialised Arabic text corrector. Use the following information to correct OCR errors:

First: common errors in this system: {confusion_context}

Second: important orthographic rules: {rules_context}

Third: correction examples: {examples_context}

Fourth: correct reference texts similar to the input: {retrieval_context}

Fifth: self-improvement notes: {insights_context}

Correct the following text. Return only the corrected text with no explanation or commentary.

Appendix B

Phase 4D Self-Reflective Insights

This appendix reproduces the complete Arabic-language diagnostic text generated by `LLMInsightsLoader.format_for_prompt()` from the PATS-A01 training-split insights file (`results/phase4d/insights/PATS-A01_insights.json`). This text is injected verbatim into the Phase 4D system prompt as the `{insights_context}` placeholder (see Appendix A.6).

The text is generated automatically from the per-error-type fix rates and introduction rates computed during Phase 4D analyse-train mode. The generation thresholds are: weakness threshold `max_fix_rate_weakness = 0.4` (fix rate below 40% qualifies as a weakness); over-correction threshold `min_intro_rate = 0.05` (introduction rate above 5% qualifies as an over-correction warning); minimum sample size `min_sample_size = 10` (error types with fewer than 10 baseline instances are excluded).

B.1 Generated Arabic Diagnostic Text

The following text was injected into the Phase 4D prompt for all 200 evaluation samples in the PATS-A01 Akhbar training split:

ملاحظات بناءً على تحليل أداء النموذج في تصحيح نصوص مشابهة:

مواطن الضعف (أخطاء متكررة لم تُصحّح — انتبه بشكل خاص):

* حذف الأحرف: يُخفق في التصحيح ٧٩٪ من الحالات

⋆ أحرف النقاط (ف\ق، ب\ت\ث، ...): يُخفق في التصحيح ٣٩٪ من الحالات

⋆ استبدالات متنوعة أخرى: يُخفق في التصحيح ٤٨٪ من الحالات

تحذير من الإفراط في التصحيح (لا تغيّر الكلمات الصحيحة):

⋆ حذف الأحرف: يُدخل أخطاء جديدة في ٢١٢٪ من الحالات

⋆ استبدالات متنوعة أخرى: يُدخل أخطاء جديدة في ٣٦١٪ من الحالات

B.2 English Translation

Notes based on an analysis of the model's performance in correcting similar texts:

Weaknesses (recurring errors that were not corrected — pay special attention):

- Character deletion: fails to correct in 97% of cases.
- Dot-group characters (fa/qaf, ba/ta/tha, ...): fails to correct in 93% of cases.
- Other miscellaneous substitutions: fails to correct in 84% of cases.

Over-correction warning (do not change correct words):

- Character deletion: introduces new errors in 212% of cases.
- Other miscellaneous substitutions: introduces new errors in 163% of cases.

B.3 Underlying Statistics

Table 2.1 reproduces the full by-type statistics from `PATS-A01_insights.json` that underlie the generated diagnostic text. Fix rate is the fraction of baseline OCR

Table 2.1: Per-type LLM performance statistics on PATS-A01 Akhbar training split (200 samples). Null entries indicate zero baseline instances.

Error Type	**Baseline**	**Fixed**	**Fix Rate**	**Intro. Rate**
Insertion	1,964	1,912	97.35%	18.13%
Deletion	32	1	3.12%	212.50%
Dot confusion	14	1	7.14%	14.29%
Other substitution	49	8	16.33%	163.27%
Alef maqsura	3	1	33.33%	100.00%
Similar shape	1	1	100.00%	800.00%
Hamza	0	—	—	—
Taa marbuta	0	—	—	—
Overall	2,063	1,924	**93.26%**	**25.06%**

errors of each type that the LLM corrected. Introduction rate is the fraction of correct tokens of each type that the LLM incorrectly modified.

The similar-shape introduction rate of 800% reflects a very small baseline (1 instance) against which 8 new similar-shape errors were introduced; this entry does not appear in the generated diagnostic text because its sample count (1) is below the `min_sample_size = 10` threshold.

Appendix C

Configuration Files

This appendix reproduces the key sections of the runtime configuration file `configs/config.yaml` used for all experiments reported in this thesis. The full file is available in the project repository.

C.1 Model and Evaluation Settings

```yaml
model:
  name: "Qwen/Qwen3-4B-Instruct-2507"
  backend: "transformers"
  temperature: 0.1
  max_tokens: 1024
  device: "auto"
  quantize_4bit: false

evaluation:
  strip_diacritics: true
  report_both: true
```

C.2 Dataset Configuration (18 datasets)

```yaml
datasets:
  - name: "PATS-A01-Akhbar-train"
```

```
    font: "Akhbar"
    type: "PATS-A01"
    pats_split: "train"
  - name: "PATS-A01-Akhbar-val"
    font: "Akhbar"
    type: "PATS-A01"
    pats_split: "validation"
  # ... (repeated for Andalus, Arial, Naskh, Simplified,
  #      Tahoma, Thuluth, Traditional -- 8 fonts x 2 splits
     )
  - name: "KHATT-train"
    split: "train"
    type: "KHATT"
  - name: "KHATT-validation"
    split: "validation"
    type: "KHATT"
```

C.3 Phase-Specific Settings

```
phase3:
  top_n: 10
  format_style: "flat_arabic"
  min_substitutions: 200

phase4:
  rules:
    format_style: "compact_arabic"
  few_shot:
    num_examples: 5
    selection: "diverse"
    max_length: 300
    min_length: 10
    max_words_changed: 15
```

```
  camel_validation:
    strategy: "revert"

phase4d:
  source_phase: "phase2"
  insights:
    max_fix_rate_weakness: 0.4
    min_intro_rate: 0.05
    min_sample_size: 10
    top_n_weaknesses: 3
    top_n_overcorrections: 2

phase5:
  corpus:
    max_sentences: 200000
    min_char_len: 30
    max_char_len: 300
    status_filter: "pri"
  retrieval:
    top_k: 3
    format_style: "numbered_arabic"

rag:
  embedding_model: >
    sentence-transformers/paraphrase-multilingual-MiniLM-
       L12-v2
  top_k: 3

phase6:
  stats:
    alpha: 0.05
    n_bootstrap: 1000

camel:
```

```
  enabled: true
  morphology:
    db: "calima-msa-r13"
    cache_size: 10000
```

Appendix D

PATS-A01 Error Examples

This appendix presents 20 representative character-level OCR error examples from the Phase 1 evaluation on PATS-A01 Akhbar training split (200 samples). Each row shows the sample identifier, the three-character context window centred on the error (with the error position in the middle), the ground-truth character (GT), the OCR-output character (OCR), the error category, and the position of the error within its word.

The context field uses Arabic text displayed right-to-left; a space character in the context indicates a word boundary. Where the OCR character is absent (deletions), the OCR column shows ∅. Where a character is spuriously inserted (insertions), the GT column shows ∅.

Table 4.1 is drawn from the `error_examples.json` output of `run_phase1.py` and covers all non-zero error categories identified in the Phase 1 taxonomy.

Observations from Examples

The dot-confusion examples in rows 1–5 illustrate the two primary visual confusion pairs identified in the Phase 1 confusion matrix: ظ/ط (both emphatic consonants with near-identical skeletons, probability 0.89) and the bidirectional ز/ر confusion (differ by a single diagonal mark). The alef maqsura examples (rows 6–8) confirm the bidirectional ى/ي confusion: both directions appear in the data, reflecting the absence of distinguishing dots in the Akhbar font rendering.

Table 4.1: Representative OCR error examples from Phase 1, PATS-A01 Akhbar training split. Context is a three-character window around the error. GT = ground truth; OCR = OCR output.

Sample	Context	GT	OCR	Category	Position
Akhbar_22	فل	ف	ق	Dot confusion	word-start
Akhbar_29	طه	ط	ظ	Dot confusion	word-start
Akhbar_57	لطه	ط	ظ	Dot confusion	word-middle
Akhbar_70	بزا	ز	ر	Dot confusion	word-middle
Akhbar_78	برز	ر	ز	Dot confusion	word-middle
Akhbar_3	بى	ى	ي	Alef maqsura	word-end
Akhbar_4	بى	ى	ي	Alef maqsura	word-end
Akhbar_88	في	ي	ى	Alef maqsura	word-end
Akhbar_7	لا	ل	ا	Similar shape	word-start
Akhbar_2	جنة	ن	∅	Deletion	word-middle
Akhbar_9	فا	ف	∅	Deletion	word-start
Akhbar_20	ال	ا	∅	Deletion	word-start
Akhbar_31	وا	و	∅	Deletion	word-start
Akhbar_56	لاة	ا	∅	Deletion	word-middle
Akhbar_4	ص ف	∅	ر	Insertion	—
Akhbar_4	غ ل	∅		Insertion	—
Akhbar_4	لاض	∅		Insertion	—
Akhbar_4	اض	∅	َ	Insertion	—
Akhbar_4	يج	∅	يَ	Insertion	—
Akhbar_1	غط	ط	ت	Other subst.	word-middle

The five deletion examples (rows 10–14) show character deletions at word-initial and word-medial positions; the deleted characters include common letters (ا, ن, و) whose absence changes the word identity while leaving a valid Arabic form. This validates the Phase 1 morphological finding that 100% of OCR errors are valid-but-wrong.

The five insertion examples (rows 15–19) are all from sample `Akhbar_4`, which is the runaway repetition artefact sample. The inserted characters include diacritics (َ, ِ) that inflate the CER count substantially; these are the errors that the Phase 2 LLM corrector resolves with a 97.35% fix rate (Section 5.1).

الملخص

يكتب هنا ملخص الرسالة

مهندس: إسم المهندس/المهندسة

تاريخ الميلاد: يوم\شهر\سنة

الجنسية: جنسية المهندس/المهندسة

تاريخ التسجيل: يوم\شهر\سنة

تاريخ المنح: ....\....\سنة

القسم: هندسة الإلكترونيات والاتصالات الكهربية

الدرجة: ماجستير العلوم

المشرفون:

ا.د. اسم المشرف الرئيسي

ا.م.د. اسم المشرف الثاني

ا.م.د. اسم المشرف الثالث

الممتحنون:

أ.د. اسم المشرف الرئيسي (المشرف الرئيسي)

أ.د. اسم الممتحن الداخلي (الممتحن الداخلي)

أ.م.د. اسم الممتحن الخارجي (الممتحن الخارجي)

(المرتبة و جهة العمل للممتحن الخارجي)

عنوان الرسالة:

عنوان الرسالة

عنوان الرسالة

إعداد

إسم المهندس/المهندسة

رسالة مقدمة إلى كلية الهندسة - جامعة القاهرة
كجزء من متطلبات الحصول على درجة
ماجستير العلوم
في
هندسة الإلكترونيات والإتصالات الكهربية

يعتمد من لجنة الممتحنين:

أ.د./أ.م.د. إسم المشرف الرئيسي المشرف الرئيسى

أ.د./أ.م.د. إسم الممتحن الداخلي الممتحن الداخلي

أ.د./أ.م.د. إسم الممتحن الخارجي الممتحن الخارجي
(المرتبة و جهة العمل للممتحن الخارجي)

كليـــة الهندســـة - جامعـــة القاهـــرة
الجيـزة - جمهوريـة مصـر العربيـــة
٢٠١٩

عنوان الرسالة

إعداد

إسم المهندس/المهندسة

رسالة مقدمة إلى كلية الهندسة - جامعة القاهرة
كجزء من متطلبات الحصول على درجة
ماجستير العلوم
في
هندسة الإلكترونيات والإتصالات الكهربية

تحت إشراف

إسم المشرف
أ.د./أ.م.د إسم المشرف الرئيسي
المرتبة
قسم هندسة الإلكترونيات والإتصالات الكهربية
كلية الهندسة - جامعة القاهرة

إسم المشرف
أ.د./أ.م.د. إسم المشرف الثاني
المرتبة
القسم
الكلية - الجامعة

إسم المشرف
أ.د./أ.م.د. إسم المشرف الثالث
المرتبة
القسم
الكلية - الجامعة

كليـة الهندسـة - جامعـة القاهـرة
الجيـزة - جمهوريـة مصـر العربيـة
٢٠١٩

عنوان الرسالة

إعداد

إسم المهندس/المهندسة

رسالة مقدمة إلى كلية الهندسة - جامعة القاهرة
كجزء من متطلبات الحصول على درجة
ماجستير العلوم
في
هندسة الإلكترونيات والإتصالات الكهربية

كليـة الهندسـة - جامعـة القاهـرة
الجيـزة - جمهوريـة مصـر العربيـة
٢٠١٩